Physics of Fluids

RESEARCH ARTICLE | NOVEMBER 12 2024

An open-source, adaptive solver for particle-resolved simulations with both subcycling and non-subcycling methods

Xuzhu Li (李虚竹) ; Chun Li (李春) ; Xiaokai Li (李晓凯) ; Wenzhuo Li (李文卓) ; Mingze Tang (唐铭泽) ; Yadong Zeng (曾亚东) ; Zhengping Zhu (朱正平)

Check for updates

Physics of Fluids 36, 113335 (2024)
https://doi.org/10.1063/5.0236509

View Online
Export Citation

Articles You May Be Interested In

An adaptive direct-forcing immersed boundary method with subcycled time advancement for compressible flows over irregular solid boundaries

Physics of Fluids (September 2026)

Highly parallelisable simulations of time-dependent viscoplastic fluid flow with structured adaptive mesh refinement

Physics of Fluids (September 2018)

An OpenFOAM solver for multiphase and turbulent flow

Physics of Fluids (April 2020)

21 September 2026 07:31:41

AIP Publishing

AIP Publishing

Physics of Fluids

Special Topics Open for Submissions

Learn More

Physics of Fluids ARTICLE pubs.aip.org/aip/pof

An open-source, adaptive solver for particle-resolved simulations with both subcycling and non-subcycling methods

Cite as: Phys. Fluids **36**, 113335 (2024); doi: 10.1063/5.0236509
Submitted: 31 August 2024 · Accepted: 23 October 2024 ·
Published Online: 12 November 2024

View Online Export Citation CrossMark

Xuzhu Li (李虚竹),[1,2] Chun Li (李春),[3] Xiaokai Li (李晓凯),[4] Wenzhuo Li (李文卓),[5] Mingze Tang (唐铭泽),[6] Yadong Zeng (曾亚东),[7,a)] and Zhengping Zhu (朱正平)[1,a)]

AFFILIATIONS

[1]Research Center for Astronomical Computing, Zhejiang Laboratory, Hangzhou 311100, China
[2]School of Mechanical Engineering, Hefei University of Technology, Hefei 230009, China
[3]School of Energy and Power Engineering, Lanzhou University of Technology, Lanzhou, Gansu 730050, China
[4]School of Physical Science and Technology, ShanghaiTech University, Shanghai 201210, China
[5]Advanced Propulsion Laboratory, Department of Modern Mechanics, University of Science and Technology of China, Hefei 230026, China
[6]School of Aeronautics, Northwestern Polytechnical University, Xi'an 710072, China
[7]Department of Computer Science, University of Texas at Austin, Austin, Texas 78712, USA

[a)]**Authors to whom correspondence should be addressed:** zengx372@utexas.edu and zhuzhp@zhejianglab.edu.cn

ABSTRACT

We present the IAMReX (incompressible flow with adaptive mesh refinement for the eXascale), an adaptive and parallel solver for particle-resolved simulations on the multi-level grid. The fluid equations are solved using a finite-volume scheme on the block-structured semi-staggered grids with both subcycling and non-subcycling methods. The particle-fluid interaction is resolved using the multidirect forcing immersed boundary method. The associated Lagrangian markers used to resolve fluid-particle interface only exist on the finest-level grid, which greatly reduces memory usage. The volume integrals are numerically calculated to capture the free motion of particles accurately, and the repulsive potential model is also included to account for the particle–particle collision. We demonstrate the versatility, accuracy, and efficiency of the present multi-level framework by simulating fluid-particle interaction problems with various types of kinematic constraints. The cluster of monodisperse particles case is presented at the end to show the capability of the current solver in handling multiple particles. It is demonstrated that the three-level AMR (Adaptive Mesh Refinement) simulation leads to a 72.46% grid reduction compared with the single-level simulation. The source code and testing cases used in this work can be accessed at https://github.com/ruohai0925/IAMR/tree/development. Input scripts and raw postprocessing data are also available for reproducing all results.

Published under an exclusive license by AIP Publishing. https://doi.org/10.1063/5.0236509

21 September 2026 07:31:41

I. INTRODUCTION

Particle-laden flows are of common occurrence in natural and industrial applications,[1–8] such as sediment transport, turbidity currents, and fluidized bed reactors. The comprehension of the physics underlying particle-turbulence interactions is crucial for these applications. For particles that are smaller than the Kolmogorov scale, point-particle simulations have provided deep insight into the interactions.[9–17] For particles that are larger than the Kolmogorov scale, particle-resolved simulations are widely utilized to study the interactions.[18–25] The immersed boundary method has become a popular approach for particle-resolved simulations due to its ability to avoid the time-consuming regeneration of Eulerian grids for moving boundaries. It typically utilizes structured Cartesian grids that are fixed in time. The no-slip boundary condition on the particle surface is satisfied implicitly by applying a volumetric forcing to the flow around the particle surface.[26–29]

Different techniques have been developed to derive the volumetric forcing. One such category is the feedback forcing technique,[30,31] the volumetric forcing is calculated by a system of virtual springs and dampers attached to the particle surface $\boldsymbol{f} = \alpha \int_0^t (\boldsymbol{u} - \boldsymbol{u}_b) d\tau + \beta(\boldsymbol{u} - \boldsymbol{u}_b)$,

Published under an exclusive license by AIP Publishing

where α and β are two free parameters and u_b is particle surface velocity. The penalty technique[32,33] can be regarded as a special instance of the feedback technique, where the parameters are set to $\alpha = 0$ and $\beta = 1/K$. The undesirable feature of the feedback technique is that those two free parameters are determined based on the flow conditions. Additionally, the characteristic time scales of the spring-damper system severely restrict the computational time step.[34,35] Another category is the direct forcing technique,[36–38] in which the particle surface is discretized using Lagrangian markers. Each marker experiences a Lagrangian interface force derived from the difference between the desired and actual velocities at the particle interface. The volumetric forcing is then calculated by spreading the Lagrangian interface force. Compared with the feedback technique, the direct forcing technique is more versatile since it eliminates stability constraints and does not require empirical constants.[34,35] However, the direct forcing technique is based on a single Lagrangian marker. When applied to multiple markers, the direct forcing on each Lagrangian marker will be affected by its neighbors, which may not enforce the no-slip boundary condition well. The multidirect forcing technique[39–41] is developed as a remedy to this problem. The no-slip boundary condition is more accurately satisfied by several applications of direct forcing via an explicit iterative procedure. In this work, we implement the multidirect forcing technique on a semi-staggered grid, which avoids the checkboard issue of the collocated grid[42,43] and can resolve multiple particles.[44]

Owing to the above advantages of the direct forcing immersed boundary (DFIB) method, it has been successfully utilized to study the interaction between thousands of particles and near-wall turbulence via particle-resolved sediment transport simulations. Ji *et al.*[45,46] investigated the statistical features of the near-wall turbulence and saltation particles in sediment transport. Kidanemariam and Uhlmann,[47,48] Kidanemariam *et al.*[49] investigated the formation of sediment patterns in sediment transport. Scherer *et al.*[50] investigated the role of turbulent large-scale motions in forming sediment patterns in sediment transport. Vowinckel *et al.*[51] investigated the mechanism of particle entertainment over an erodible bed. Zhu *et al.*[52] investigated the probability distribution functions of several saltation parameters in sediment transport. Jain *et al.*[53] investigated the sediment transport with non-spherical particles. Although particles accumulate near the sediment bed in sediment transport, those simulations employed uniform grids across the entire computational domain to resolve not only the sediment bed but also the particle-free region further away from it, resulting in a huge amount of computation.

Great efforts were made to reduce the number of Eulerian grids and Lagrangian markers required by the DFIB method. The DFIB method employs a Dirac delta function to interpolate the fluid velocity from an Eulerian grid onto Lagrangian markers as well as spreading the forcing in the opposite direction. The commonly used Dirac delta function proposed by Roma *et al.*[54] requires uniform Eulerian grids to conserve total force and torque, which substantially increases the number of Eulerian grids, especially for the channel flow. Because the turbulence scale near the wall is much smaller than that in the channel center, by employing the reproducing kernel particle method (RKPM)[55] to modify the Dirac delta function, the DFIB method can be applied to non-uniform Eulerian grids, significantly reducing the Eulerian grids required by the DFIB method.[56–58] Furthermore, Akiki and Balachandar[57] proposed a dynamic non-uniform distribution of Lagrangian markers on a sphere, which resulted in a 76.96% reduction in the number of Lagrangian markers compared with uniform distribution in the particle-resolved simulation of 640 monodisperse spherical particles randomly distributed in the channel flow.

In addition to the non-uniform mesh approach described above, another idea to reduce the Eulerian cells requirement is to employ a multi-level grid and utilize the AMR technique for the DFIB method. AMR is a highly effective computational technique for tackling the complexities of fluid flows.[59,60] It stands out for its ability to dynamically adjust grid resolution based on the evolving solution, optimizing computational resources precisely where they are most needed. For the particle-resolved simulation, one can refine the grid cells near the particle interface and/or at the wake of particles where the velocity gradient is large. Unlike static mesh refinement, which maintains a fixed grid hierarchy, AMR can refine and coarsen the grid as needed. There are three primary types of AMR. The first is cell-based refinement, where each cell that meets refinement criteria is divided into four (in 2D) or eight (in 3D) smaller cells, organized in a quad- or oct-tree structure.[61–63] Libraries such as p4est[64] and libMesh[65] efficiently support this approach. The second type is patch-based refinement, which generalizes the cell-based method but requires logically rectangular regions, often called grids, patches, or boxes. This strategy, also known as quad-tree or oct-tree patch-based refinement, constructs fine patches of a minimum size in each dimension. The FLASH library[66] supports this type. The third type, also patch-based, organizes data into levels of refinement based on mesh resolution. Unlike tree-structured methods, this approach constructs variable-sized patches that are logically rectangular, which makes it relatively easy to use the domain decomposition method for parallelization.[67] Equations on the nested patches can also be solved efficiently utilizing the multigrid (MG) solver.[44] A number of open-source libraries, such as AMReX (Adaptive Mesh Refinement for the eXascale),[68,69] ForestClaw,[70] and Chombo,[71] support this approach. We note the last two types of AMR also fall into the category of two "block-structured refinement." In this work, we exclusively investigate the third type of AMR and build our adaptive solver on a block-structured framework.[68,69]

Some previous studies have explored the combination of AMR with particle-resolved simulations. For instance, Bhalla *et al.*[72] integrated the distributed Lagrange multiplier (DLM) immersed boundary (IB) method with block-structured AMR, demonstrating the accuracy of their approach through test cases involving a single particle in the single-phase flow. Zeng and Shen[73] and Zeng *et al.*[74] also incorporated the DLM algorithm within a collocated AMR grid framework. However, their validation was limited to single-particle scenarios. Compared with the generation of markers in Refs. 38, 40, and 52, a known limitation of the DLM algorithm is that Lagrangian markers must be placed within all particles, leading to increased computational costs and memory usage. Additionally, Bhalla *et al.*[75] simulated the dielectrophoretic motion of particles in microfluidic channels, while Li and Kong[76] combined tree-structured AMR with unstructured grids to simulate spray particles in multiphase flows. Nangia *et al.*[77] treated a point absorber (a type of wave energy converter) as a particle and added a spring-damping system to study its energy absorption efficiency in waves. However, in those studies, the flow solutions were updated using a composite time-stepping approach, where the discretized equations for velocity and pressure were coupled across coarse-fine grid boundaries and solved simultaneously at multiple levels. This

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

coupling constrained the time step to the finest grid spacing to maintain numerical stability.

In this work, we develop an adaptive AMR framework that allows level-by-level advancement, using both subcycling and non-subcycling methods, for particle-resolved simulations. For the subcycling method, it advances the flow field using different time steps since the coarser grid size allows using a large time step size. While for the non-subcycling method, the flow field on all levels is advanced using the same time step due to the constraint of Courant–Friedrichs–Lewy (CFL) number on the finest level.[44,78] To provide a more intuitive understanding of subcycling and non-subcycling methods, we can draw an analogy to pedagogical practices. The subcycling method is analogous to a teacher tailoring educational content and pacing to students with diverse learning capacities. In this context, coarse grids, akin to high-aptitude students, can progress more rapidly with an accelerated learning schedule. Conversely, fine grids, comparable to students requiring additional time for knowledge assimilation, necessitate more temporal resources (manifested as additional time steps). In contrast, the non-subcycling method is reminiscent of a standardized curriculum where high-performing students must, to some extent, accommodate their peers with different learning rates. This scenario parallels the uniform time step progression across both coarse and fine grids. In addition, we note that both subcycling and non-subcycling methods belong to a level-by-level temporal advancement paradigm and necessitate synchronization operations to ensure congruence between results at different levels.[42,43,73,79] Section III C 1 provides a comprehensive comparative analysis of these two methods, elucidating their respective characteristics, advantages, and limitations. Furthermore, since an AMR framework for particle-resolved simulations involving multiple particles is still lacking, developing such an open-source framework would improve the capability of the DFIB method and help us to have a deeper comprehension of interactions between multiple large particles and turbulence.

The focus of the present paper is to develop an adaptive level-by-level AMR framework for the DFIB method, which greatly reduces its Eulerian grid cells and the computational time for particle-resolved simulations. Our open-source framework can efficiently simulate multiple particles within the flow field using either subcycling or non-subcycling methods. The remainder of this paper is organized as follows: we start with the mathematical formulation of the fluid-particle system in Sec. II, including the operators used in the Lagrangian–Eulerian interaction. Next, both the single-level and multi-level advancement algorithms are presented in Sec. III. We first describe the numerical discretization of the single level in Sec. III A, in which different types of kinematic constraints are considered (Sec. III B). For the multi-level time advancement in Sec. III C, we compare the subcycling and non-subcycling methods (Sec. III C 1) and highlight the benefits of the synchronization operations (Sec. III C 2). We then briefly introduce our open-source framework IAMReX in Sec. III D. The particle-related validation cases that highlight the accuracy, efficiency, and robustness of our adaptive solver are then given in Sec. IV. Finally, the conclusions and future work are given in Sec. V.

II. MATHEMATICAL FORMULATION

This section describes the governing equations for a fluid-particle system occupying a three-dimensional multi-level Cartesian grid $\Omega \subset \mathbb{R}^3$. The upper-left part of Fig. 1 shows a schematic of two particles on a three-level grid with AMR. When a schematic is sliced, the particles can be seen distributed on the finest level from the bottom-left corner of Fig. 1. The momentum and material incompressibility equations are described using a fixed Eulerian coordinate system $\mathbf{x} = (x_1, x_2, x_3) \in \Omega$. The markers attached to the particle are

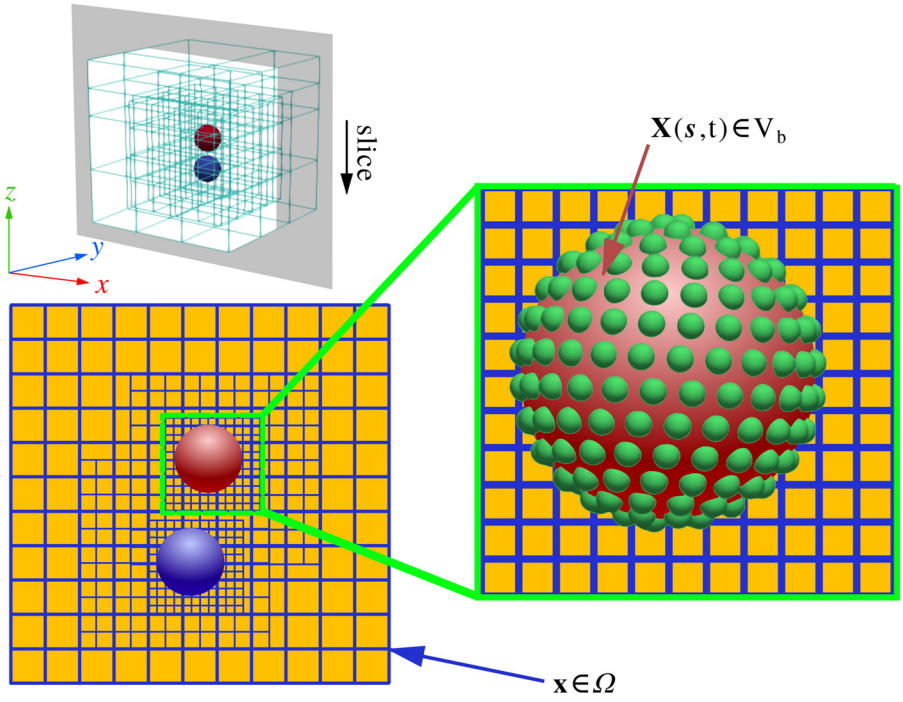


FIG. 1. Upper-left: schematic of two solid particles on a three-dimensional multi-level Cartesian grid. Bottom left: a slice of the upper-left schematic. Right: schematic of Eulerian grid cells and Lagrangian markers. The Eulerian grid cells (■, orange) discretize the Ω region, and the Lagrangian markers (○, green) discretize the $V_b(t)$ region.

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

described using a Lagrangian coordinate system, where $\mathbf{s} = (s_1, s_2, s_3) \in \Omega_c$ denotes the fixed material coordinate system attached to the structure and $\Omega_c \subset \mathbb{R}^3$ is the Lagrangian curvilinear coordinate domain. The position of the particle at time t is $\mathbf{X}(\mathbf{s}, t)$; it occupies a volumetric region $V_b(t) \subset \Omega$. The equations of motion of the coupled fluid-particle system are as follows:

$$\rho_f\left(\frac{\partial \mathbf{u}}{\partial t}(\mathbf{x},t) + \nabla\cdot(\mathbf{u}(\mathbf{x},t)\mathbf{u}(\mathbf{x},t))\right) = -\nabla p(\mathbf{x},t) + \nabla\cdot[\mu_f\left(\nabla\mathbf{u}(\mathbf{x},t) + \nabla\mathbf{u}(\mathbf{x},t)^T\right)] + \rho_f\mathbf{g} + \mathbf{f}_c(\mathbf{x},t), \tag{1}$$

$$\nabla\cdot\mathbf{u}(\mathbf{x},t) = 0, \tag{2}$$

$$\mathbf{f}_c(\mathbf{x},t) = \int_{V_b(t)} \mathbf{F}_c(\mathbf{s},t)\delta(\mathbf{x}-\mathbf{X}(\mathbf{s},t))\mathrm{d}\mathbf{s}, \tag{3}$$

$$\frac{\partial \mathbf{X}}{\partial t}(\mathbf{s},t) = \mathbf{U}(\mathbf{s},t), \tag{4}$$

$$\mathbf{U}(\mathbf{s},t) = \int_{V_b(t)} \mathbf{u}(\mathbf{x},t)\delta(\mathbf{x}-\mathbf{X}(\mathbf{s},t))\mathrm{d}\mathbf{x}. \tag{5}$$

Here, $\mathbf{u}(\mathbf{x}, t)$ is the Eulerian velocity of the coupled fluid-particle system, $p(\mathbf{x}, t)$ is the pressure, ρ_f is the Eulerian density field, and μ_f is the dynamic viscosity of the fluid-structure system. The gravitational acceleration is written as $\mathbf{g} = (g_1, g_2, g_3)$. In Eq. (1), $\mathbf{f}_c(\mathbf{x}, t)$ represents the Eulerian force density, which accounts for the presence of the solid in the domain. $\delta(\mathbf{x}) = \Pi_{i=1}^3\delta(x_i)$ represents the three-dimensional Dirac delta function, which is employed to exchange the information between the Eulerian quantity and Lagrangian quantity. Specifically, Eq. (3) converts the Lagrangian force density $\mathbf{F}_c(\mathbf{s}, t)$ to an equivalent Eulerian force density $\mathbf{f}_c(\mathbf{x}, t)$, in an operation that is referred to as *force spreading*. Equation (5) maps the Eulerian velocity $\mathbf{u}(\mathbf{x}, t)$ to the Lagrangian marker velocity $\mathbf{U}(\mathbf{s}, t)$, in an operation that is referred to as *velocity interpolation*. For notational convenience, we denote the force spreading operation in Eq. (3) as

$$\mathbf{f}_c = \mathscr{S}[\mathbf{X}]\mathbf{F}, \tag{6}$$

where $\mathscr{S}[\mathbf{X}]$ is the force spreading operator. Similarly, the velocity interpolation operation in Eq. (5) is written in shorthand notation as

$$\mathbf{U} = \mathscr{J}[\mathbf{X}]\mathbf{u}, \tag{7}$$

where $\mathscr{J}[\mathbf{X}]$ is the velocity interpolation operator. We note that *force spreading* and *velocity interpolation* work together to satisfy no-slip boundary conditions at the fluid-solid interface. As shown in Refs. 77 and 80, these two coupling operators also conserve energy as long as $\mathscr{S}$ and $\mathscr{J}$ are adjoint.

III. NUMERICAL DISCRETIZATION

This section presents the numerical discretization of Eqs. (1)–(5). The discretization of the fluid system on a single level is elucidated in Sec. III A, followed by an examination of two distinct types of kinematic constraints in Sec. III B. Subsequently, Sec. III C delineates the discretization and advancement procedures on a multi-level grid with adaptive mesh refinement (AMR), incorporating both subcycling and non-subcycling methodologies. The discourse concludes with an introduction to the open-source code IAMReX in Sec. III D.

A. Single-level advancement

To solve the partial differential equations of Eqs. (1)–(5), the canonical projection[44,81] is applied to the semi-staggered grid. As shown in Fig. 2, the fluid velocity (u and v), the Eulerian force f, and particle volume fraction α are located at the cell center. The pressure p and level set function ϕ are at the node center. The temporal and spatial discretizations of equations for single-level advancement are considered here. At the time t^n, the Eulerian velocity $\mathbf{u}^n$ and pressure $p^{n-1/2}$ are known. The particle position $\mathbf{X}_l^n$ and velocity $\mathbf{U}(\mathbf{X}_l^n)$ are also available. The time advancement during the interval $[t^n, t^{n+1}]$ proceeds as follows.

1. Step 1

The intermediate velocity $\tilde{\mathbf{u}}^{*,n+1}$ is solved semi-implicitly as

$$\rho_f\left(\frac{\tilde{\mathbf{u}}^{*,n+1}-\mathbf{u}^n}{\Delta t} + \nabla\cdot(\mathbf{u}\mathbf{u})^{n+\frac{1}{2}}\right) = -\nabla p^{n-\frac{1}{2}} + \frac{1}{2}\left(\nabla\cdot\mu\nabla\tilde{\mathbf{u}}^{*,n+1} + \nabla\cdot\mu\nabla\mathbf{u}^n\right) + \rho_f\mathbf{g}, \tag{8}$$

where the convective term $\nabla\cdot(\mathbf{u}\mathbf{u})^{n+\frac{1}{2}}$ is calculated using the second-order Godunov scheme. The Godunov algorithm has been proven to be a very robust method for handling conservation laws, especially for high Reynolds number flows. Many researchers who previously worked on incompressible fluid simulations have applied this method to deal with the convection terms in equations.[44,82,83] In our previous work,[74,84] we provided a detailed derivation of the Godunov algorithm. In our recent work,[78] we applied this algorithm to simulate two-phase flows at high Reynolds numbers (i.e., $Re \approx 10^6$) and obtained stable and physically meaningful results. In this step, only the pure fluid system is solved and no particle-related influence is included.

2. Step 2

The updated velocity $\tilde{\mathbf{u}}^{*,n+1}$ needs to be corrected to satisfy the no-slip boundary condition at the fluid–particle interfaces $\partial V_b(t)$.

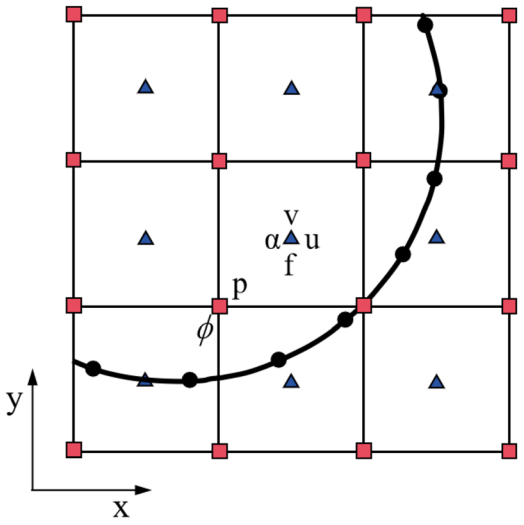


FIG. 2. Sketch of the two-dimensional semi-staggered grid and variable locations. The blue triangles, red squares, and black circles represent cell-centered variables, node-centered variables, and interface Lagrangian markers.

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

This step is divided into four substeps[40,41] in Algorithm 1. We first interpolate the intermediate Eulerian velocity obtained from *step 1* to the Lagrangian velocity of markers. The Lagrangian forces are then calculated based on the desired velocity at the interface and the intermediate velocity. Next, the Eulerian forces are obtained from the spreading of Lagrangian forces by using either the three-point or four-point delta function.[40,41,74] As shown in Fig. 3, the Eulerian cells, enclosed by red dashed circles, refer to grid areas that are influenced by two blue markers. These two Lagrangian markers also share some intersected areas, which are marked by green arrows. Finally, the Eulerian velocity is corrected by the updated Eulerian force.

In Algorithm 1 mentioned above, four points need to be noted. First, the Lagrangian markers only exist on the finest level during the Eulerian–Lagrangian interaction process. This brings the benefits of memory saving since particle-related information does not need to be stored on coarser levels. Second, the Lagrangian markers only distribute on the surface of particles. This is different from the DLM method in Refs. 72 and 77, in which the markers also appear inside the particle and there is one marker per Eulerian grid cell. Figure 4 shows how the number of markers changes with the d/h for a single particle scenario. It is seen that as d/h increases, the present needs much fewer markers compared with the DLM method. Third, the multi-direct forcing algorithm includes an outer loop with m ranging from 1 to N_s, which controls the degree of coupling between Eulerian and Lagrangian variables. The original method of Uhlmann[38] corresponds to the case of $N_s = 0$. Increasing N_s can not only enhance their coupling but also increase the computational load. Based on the experience in the previous work[40,41] and tests presented in this paper, it is sufficient to set N_s to 2–3 for all cases in Sec. IV. Finally, if a system has multiple particles, each particle goes into 1 to N_s loop sequentially. The corrected Eulerian forces we employ take into account the effects of all particles. This consideration also applies to the calculation of the particle volume fraction (PVF) field in Sec. IV A.

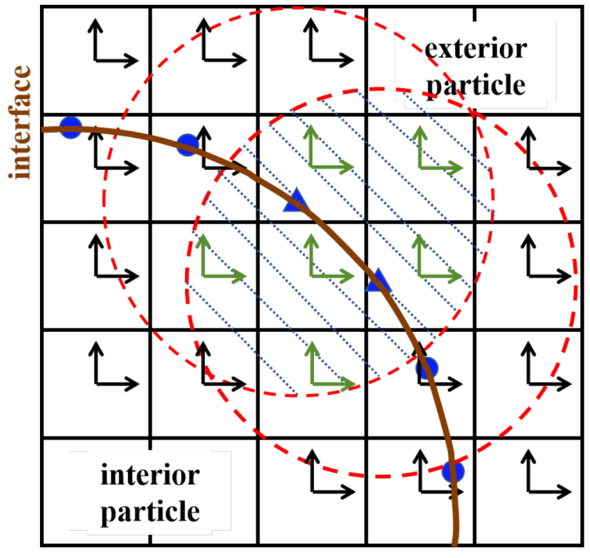


FIG. 3. Illustration of the diffuse distribution of the IBM force around the particle interface.

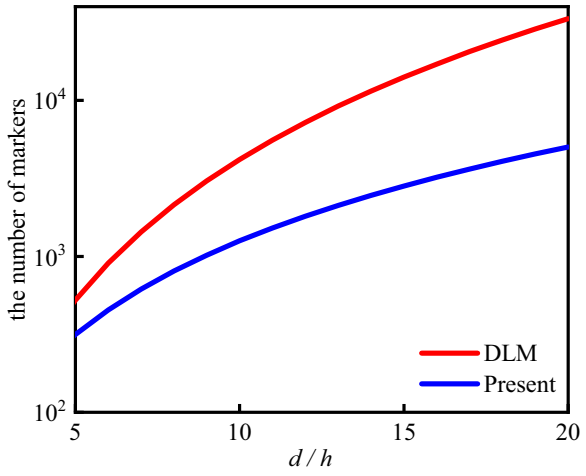


FIG. 4. Number of markers change with the d/h for a single particle scenario, where d is the particle diameter and h is the grid spacing.

ALGORITHM 1. Multidirect forcing method for fluid-particle interaction.

$\mathbf{u}^{(0)} = \tilde{\mathbf{u}}^{*,n+1}$
for $m = 1$ **to** N_s **do**
  Interpolate Lagrangian Velocity,
  $\mathbf{U}^{m-1}(\mathbf{X}_l^n) = \sum_{i=1}^{N_x}\sum_{j=1}^{N_y}\sum_{k=1}^{N_z}\mathbf{u}^{(m-1)}(\mathbf{x}_{i,j,k})\delta_h(\mathbf{x}_{i,j,k} - \mathbf{X}_l^n)h^3$
  Calculate Lagrangian Force,
  $\mathbf{F}^m = \mathbf{F}^{m-1} + (\mathbf{U}^d(\mathbf{X}_l^n) - \mathbf{U}^{m-1}(\mathbf{X}_l^n))/\Delta t$
  Spreading Lagrangian Force onto Eulerian Force,
  $\mathbf{f}^m(\mathbf{x}_{i,j,k}) = \sum_{l=1}^{N_l}\mathbf{F}^m(\mathbf{X}_l)\delta_h(\mathbf{x}_{i,j,k} - \mathbf{X}_l)\Delta V_l$
  Correct Eulerian Velocity,
  $\mathbf{u}^{(m)} = \mathbf{u}^{(0)} + \Delta t\mathbf{f}^m(\mathbf{x}_{i,j,k})$
end for
$\mathbf{u}^{*,n+1} = \mathbf{u}^{(m)}$

3. Step 3

With the updated intermediate velocity $\mathbf{u}^{*,n+1}$ in *step 2*, a level projection operator is applied to obtain the updated pressure $p^{n+1/2}$ and velocity $\mathbf{u}^{n+1}$ fields. An auxiliary variable $\boldsymbol{V}$ is first calculated by

$$\boldsymbol{V} = \frac{\mathbf{u}^{*,n+1}}{\Delta t} + \frac{1}{\rho_f}\nabla p^{n-\frac{1}{2}}. \tag{9}$$

Then, $\boldsymbol{V}$ is projected onto the divergence-free velocity field to obtain the updated pressure $p^{n+1/2}$ via

$$L_{\rho_f}^{cc,l}p^{n+1/2} = \nabla\cdot\boldsymbol{V}, \tag{10}$$

where $L_{\rho_f}^{cc}p^{n+1/2}$ is the density-weighted Laplacian operator to $\nabla\cdot(1/\rho_f\nabla p^{n+1/2})$.[44,84] Finally, the divergence-free velocity $\mathbf{u}^{n+1}$ on level l is obtained as

$$\mathbf{u}^{n+1} = \Delta t\left(\boldsymbol{V} - \frac{1}{\rho_f}\nabla p^{n+1/2}\right). \tag{11}$$

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

The projection is stable and appears to be well behaved in various numerical tests[85,86] and practical applications.[42,82]

4. Step 4

After completing *step 3*, we obtain the divergence-free fluid velocity at t^{n+1}. The particle-related information also needs to be updated from t^n to t^{n+1}. Depending on different kinematic constraints, the particle motion is categorized into prescribed motion and free motion. The specific updates are detailed in Sec. III B.

B. Types of kinematic constraints

1. Prescribed motion

If the motion of the particle is prescribed, then its velocity and position are known *a priori* and not influenced by the surrounding fluid. Thus, the centroid position $\mathbf{X}_r^n$, centroid velocity $\mathbf{U}_r^n$ at t^n, centroid velocity $\mathbf{U}_r^{n+1}$ at t^{n+1}, and angular velocity $\mathbf{W}_r^n$ of the body are given. The desired velocity $\mathbf{U}^d(\mathbf{X}_l^n)$ of the markers in Algorithm 1 is calculated as

$$\mathbf{U}^d\left(\mathbf{X}_l^n\right) = \mathbf{U}_r^n + \mathbf{W}_r^n \times \mathbf{R}_l^n, \tag{12}$$

where $\mathbf{R}_l^n = (\mathbf{X}_l^n - \mathbf{X}_r^n)$. The new position of the centroid of the particle $\mathbf{X}_r^{n+1}$ is updated using the midpoint scheme as

$$\mathbf{X}_r^{n+1} = \mathbf{X}_r^n + \frac{\Delta t}{2}(\mathbf{U}_r^{n+1} + \mathbf{U}_r^n). \tag{13}$$

2. Free motion

In contrast to the prescribed kinematics case, the motion of a freely moving particle is influenced by the surrounding fluid. To account for this two-way interaction, the following governing equations of the particle systems are solved:[40,41]

$$\rho_p V_p \frac{d\mathbf{U}_r}{dt} \approx -\rho_f \sum_{l=1}^{N_L} \mathbf{F}_l^{n+1/2} \Delta V_l + \rho_f \frac{d}{dt}\left(\int_{V_p} \mathbf{u} dV\right) + (\rho_p - \rho_f) V_p \mathbf{g} + \mathbf{F}_c^{n+1/2}, \tag{14}$$

$$I_p \frac{d\mathbf{W}_r}{dt} \approx -\rho_f \sum_{l=1}^{N_L} \mathbf{R}_l^n \times \mathbf{F}_l^{n+1/2} \Delta V_l + \rho_f \frac{d}{dt}\left(\int_{V_p} \mathbf{r} \times \mathbf{u} dV\right) + \mathbf{T}_c^{n+1/2}. \tag{15}$$

On the right-hand side of Eqs. (14) and 15, the term $\mathbf{F}_l^{n+1/2}$ refers to the Lagrangian force, coming from the final value of $\mathbf{F}^m$ in Algorithm 1. The time derivatives of momentum integration $\rho_f \frac{d}{dt}(\int_{V_p} \mathbf{u} dV)$ and angular momentum integration $\rho_f \frac{d}{dt}(\int_{V_p} \mathbf{r} \times \mathbf{u} dV)$ within the particle are also included. These two integrated terms account for flow unsteadiness by using the PVF field (Sec. IV A). The term $(\rho_p - \rho_f) V_p \mathbf{g}$ considers the buoyancy effects. The terms $\mathbf{F}_c^{n+1/2}$ and $\mathbf{T}_c^{n+1/2}$ refer to the induced force and torque generated by the particle collision, respectively. If there is only one single particle in the system, both $\mathbf{F}_c^{n+1/2}$ and $\mathbf{T}_c^{n+1/2}$ are set to be zero. On the left-hand side of Eqs. (14) and (15), we use the second-order mid-point scheme to integrate particle motions.[52] After updating the particle centroid velocity $\mathbf{U}_r^{n+1}$ and angular velocity $\mathbf{W}_r^{n+1}$ at t^{n+1}, we go back to Eq. (13) to update new position of the particle centroid $\mathbf{X}_r^{n+1}$.

The time advancement scheme in this work is not fully implicit,[52] yet it can deal with the free motion applied to particles either with a large density ratio (i.e., $\frac{\rho_p}{\rho_f} \geq 10$) or a small density ratio (i.e., $\frac{\rho_p}{\rho_f} \approx 1.5 - 2$).[40,41] We found it is robust and fast enough to handle all the testing cases in Sec. IV. Before ending this section, we also emphasize that our method is similar to the "weak coupling" method used in the sharp-interfaced immersed boundary method, which requires only one solution for fluid and solid solver during each time step and no iterations are needed between these two solvers.[87,88] It is easier to extend the current portable solver to the "strong coupling" method, which then re-projects the flow part, re-updates the solid particle, and performs a convergence checking between the fluid solver and the solid solver during each sub-iteration.[74,89]

C. Multi-level advancement

In this work, we use a level-by-level time advancement method[78,84] to advance the fluid and particle solution on the multi-level grid within the BSAMR framework. Specifically, we introduce both the subcycling method and the non-subcycling method (Sec. III C 1). Finally, we describe several synchronization operations to better achieve the composite solution (Sec. III C 2).

1. Subcycling and non-subcycling methods

To advance variables on a multi-level grid, we utilize both the subcycling and non-subcycling methods with a level-by-level approach. In the subcycling method, variables on different levels progress with distinct time step sizes. The primary advantage of this approach is that maintaining the Courant–Friedrichs–Lewy (CFL) number constant across grid levels allows for larger time steps on coarser grids due to their larger spacing.[44,85] With a refinement ratio of two between adjacent levels, the time step size on the coarser level Δt^l can be twice as large as that on the finer level Δt^{l+1}. Conversely, in the non-subcycling method, variables on all levels advance with the same time step size determined by the finest level $l_{\max}$. Fig. 5 schematically shows how the subcycling and non-subcycling methods are used to advance the variables on a multi-level grid with n levels.

To provide a more intuitive understanding of subcycling and non-subcycling methods in the context of AMR, let us consider an illustrative example. Suppose we have a three-level grid hierarchy comprising levels 0, 1, and 2, and our objective is to advance one time step on the coarsest level. In the subcycling method, the time step on a coarser grid is twice that of its adjacent finer grid. Consequently, we have the relationship $\Delta t^0 = 2\Delta t^1 = 4\Delta t^2$. As level 0 advances by one step, level 1 must progress two steps, and level 2 must advance four steps. In total, this subcycling method needs seven advancement steps. Conversely, in the non-subcycling method, levels 0, 1, and 2 all advance using the time step Δt^2 on the finest level. Therefore, to progress one time step on the coarsest level, the non-subcycling method requires a total of $3 \times 4 = 12$ steps.

Although the subcycling method takes fewer steps compared with the non-subcycling method, there are some tradeoffs between the subcycling and non-subcycling methods beyond the computational efficiency. First, the subcyling method takes more effort to implement and one has to design a recursive algorithm to loop all levels. When the subcycling method is combined with the synchronization operations (Sec. III C 2), it brings more complexities since the refluxing operations need to be applied every time a finer level catches up with a coarser level.[42–44] On the other hand, one can design the non-cycling algorithm in a sequential way by looping all levels one-by-one. The non-subcycling method also benefits because all levels already achieve the same time point before

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

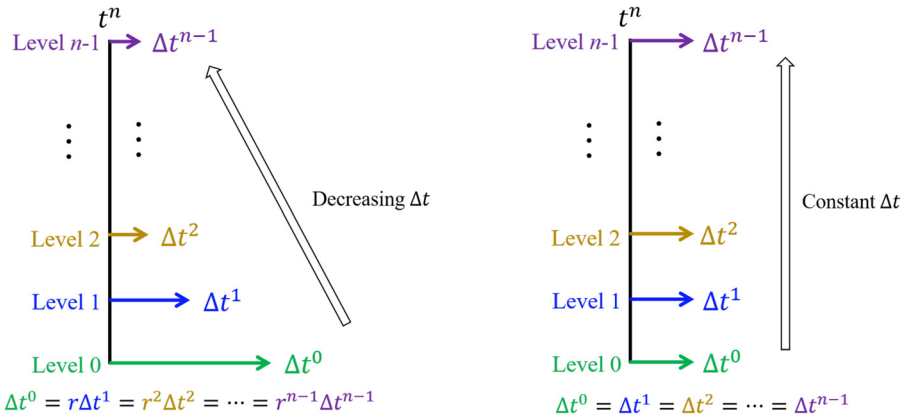


FIG. 5. Schematic of the substeps in the level-by-level advancement method for a n-level grid. Left: the subcycling method. Right: the non-subcycling method. The parameter $r = 2$ is the refining ratio between two consecutive levels.

applying the synchronization operations. Second, the subcycling method needs both spatial and temporal interpolations, while the non-subcycling method only needs temporal interpolations. In our level-by-level time advancement, the finer level needs the coarser level data to fill its ghost cell values and satisfy the boundary conditions. Thus, in the subcycling method, the data at t^n and t^{n+1} on the coarser level first interpolate to the data at $t^{n+1/2}$ and then the spatial interpolation is applied using both the interpolated data $t^{n+1/2}$ and data on the finer level. Third, the subcyling method is not always working in the practical use. In our previous work,[74] we apply the subcycling method to generate the stokes waves on a three-level AMR grid. We found that the simulation immediately blows up at the first iteration because a large time step on the coarsest level disturbs the entire air-water system and generates a large velocity gradient at the inlet. We then use the non-subcycling method at the first several hundred iterations and the switch back to the subcycling method when the simulation reaches the quasi-steady state. This switching trick helps to stabilize the solving process and can provide a physically reasonable solution. In the future, we can even dynamically identify the flow features and design some criteria to guide the switching between the subcycling and the non-subcycling method. Finally, most of the current AMR-related packages utilize the non-subcycling method, including IBAMR (https://github.com/IBAMR/IBAMR.git), basilisk (http://basilisk.fr/), and incflo (https://github.com/AMReX-Fluids/incflo.git). It is recommended that the developers start with the non-subcycling method for utilizing the AMR features because of easier implementation. After validating the accuracy of the non-subcycling method, one can gradually improve the codes to consider the subcycling method for better computational efficiency.

In this work, we utilize both the subcycling and the non-subcycling methods for simulations of resolved particles. It should be noted that both of these methods produce consistent and accurate results for the single particles and multiple particle cases in Sec. IV. Within the level-by-level framework, the non-subcycling method is relatively easier to implement and no temporal recursive procedure is involved. The subcycling method, on the other hand, allows large time steps on the coarser levels and thus reduces the overall computational cost.

2. Synchronization

The synchronization operations are used to make the solution data consistent across all levels.[42–44] There are three substeps of synchronization operations after the level advancement. First, the flow velocity **u** and pressure p on coarser levels are replaced by the corresponding averaging value on the finer levels. There is no need to average the particle-related variables since they only exist on the finest level. Second, we use a refluxing operation to account for an imbalance of the momentum and scalar fluxes at the coarse fine (CF) boundary.[74,84] Our previous work[84] has validated that the refluxing operation can help add the mass and momentum conservation for tracer advection and double shear layer problems. Last, a composite grid projection is applied to enforce the divergence-free condition on the velocity field across the entire hierarchy.[42,44]

3. Summary of the multi-level advancement

The synchronization operations are used to make the solution data consistent across all levels.[42–44] There are three substeps of synchronization operations after the level advancement. First, the flow velocity **u** and pressure p on coarser levels are replaced by the corresponding averaging value on the finer levels. There is no need to average the particle-related variables since they only exist on the finest level. Second, AMR requires numerical methods to deal with the coarse fine (CF) boundary interface where cells in different refinement levels meet. We thus use a refluxing operation to account for an imbalance of the momentum and scalar fluxes at the CF boundary.[74,84] Our previous work[84] has validated that the refluxing operation can help add the mass and momentum conservation for tracer advection during vortex merging and double shear layer problems. Last, a composite grid projection is applied to enforce the divergence-free condition on the velocity field across the entire hierarchy.[42,44]

Algorithm 2 summarizes the adaptive multi-level advancement framework using both subcycling and non-subcycling methods. After initializing flow-related variables on all levels and particle-related variables on the finest level, time advancement can proceed using either method. Synchronization occurs when a coarser level catches up with a finer level.

As a final remark, we emphasize that our multi-level advancement algorithm employs a level-by-level approach, distinct from the composite advancement method.[72,82,90] In the level-by-level method, each level's variables advance independently until synchronization, reducing time step constraints on coarser levels. Conversely, the composite advancement method uses composite variables for time advancement, relying only on variables in non-overlapping regions. This makes it less flexible

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

to integrate both subcycling and non-subcycling methods. Our level-by-level approach, however, handles both methods with ease.

ALGORITHM 2. Multi-level advancement.

Initialize $\mathbf{X}_r^0$, $\mathbf{U}_r^0$, $\mathbf{W}_r^0$, $\mathbf{u}^0$, and p^0 on level 0
$l \leftarrow 0$
while refinement criteria are satisfied on level l and $l < l_{\max}$ **do**
  Regrid the grid to level $l+1$
  Initialize $\mathbf{X}_r^0$, $\mathbf{U}_r^0$, $\mathbf{W}_r^0$, $\mathbf{u}^0$, and p^0 on level $l+1$
  $l \leftarrow l+1$
end while
Initialize $\mathbf{X}_l^0$, $\mathbf{U}_l^0$ and $\mathbf{R}_l^0$ for all Lagrangian markers on level $l_{\max}$
if subcycling method is used **then**
  $\Delta t^l = 2^{l_{\max}-l}\Delta t^{l_{\max}}$ for all $0 \le l < l_{\max}$
else
  $\Delta t^l = \Delta t^{l_{\max}}$ for all $0 \le l < l_{\max}$
end if
for $n = 1, n_{\max}$ **do** ▷ $n_{\max}$ is the number of time steps in the simulation
  SINGLE-LEVEL-ADVANCEMENT$(0, t_n^0, t_n^0 + \Delta t^0, \Delta t^0)$
  Apply the synchronization projection [74, 78, 84]
  Refine the grid and interpolate $\mathbf{u}$ and p onto new levels
end for

procedure SINGLE-LEVEL-ADVANCEMENT$(l, t^l, t_{\max}^l, \Delta t^l)$
  while $t^l < t_{\max}^l$ **do**
    Solve momentum Eq. (8) to obtain $\tilde{\mathbf{u}}^{*,n+1}$
    Apply the Algorithm 1 for the fluid-particle coupling and obtain the intermediate velocity $\mathbf{u}^{*,n+1}$
    Apply the level projection using Eqs. (9)–(11) to obtain the updated pressure $p^{n+1/2}$ and velocity $\mathbf{u}^{n+1}$ fields
    Update particle-related information from t^n to t^{n+1} based on different constraints
    if $l < l_{\max}$ **then**
      SINGLE-LEVEL-ADVANCEMENT$(l+1, t^l, t^l + \Delta t^l, \Delta t^{l+1})$
    end if
    $t^l \leftarrow t^l + \Delta t^l$
  end while
  if $l > 0$ **then**
    Average all data from finer levels onto coarser levels[74,78,84]
  end if
  if $l < l_{\max}$ **then**
    Perform the refluxing operation[74,78,84]
  end if
end procedure

D. IAMReX framework

We extend the AMReX-based[68,69] application IAMR (incompressible flow with adaptive mesh refinement)[44,85] to a much more powerful framework IAMReX. In the IAMReX, the Navier–Stokes equations are solved on a semi-staggered multi-level grid using the projection method.[44] The gas-liquid interface is captured using either the level set (LS) method or conservative level set (CLS) method.[78,84] The fluid-particle interface is resolved using the multidirect forcing immersed boundary method.[41,52] IAMReX is a publicly accessible platform designed specifically for developing massively parallel block-structured adaptive mesh refinement (BSAMR) applications. The code now supports hybrid parallelization using either pure MPI or MPI+OpenMP for multicore machines.[68] The source code for IAMReX, testing cases used in this work can be accessed at https://github.com/ruohai0925/IAMR/tree/development. All input scripts and raw postprocessing data are uploaded to https://pan.baidu.com/s/1bZRoDunjBv7bqYL8CI3ASA?pwd=i5c2 for interested readers to reproduce the results in Sec. IV.

Many downstream applications based on the AMReX framework have detailed discussions on scalability.[5,68,91,92] Yao *et al.*[91] discussed the parallel efficiency of ARTEMIS (Adaptive mesh Refinement Time-domain ElectrodynaMIcs Solver) and proved effective scalability on manycore and GPU-based supercomputing architectures. Sharma *et al.*[92] developed the AMR-Wind, which is an open-source block-structured adaptive grid incompressible-flow solver with good efficiency and scalability in parallel computing processes. In one of our previous works,[5] we conducted parametric studies to investigate the computational efficiency of incompressible flows on a block-structured adaptive mesh. The parameters included refining block size, refining frequency, maximum level, and cycling method.

In this work, we use different parallelization strategies for the Eulerian grid and Lagrangian points while performing particle-resolved simulations. This approach is called the dual-grid algorithm, and details can be found in Pozzetti *et al.*[93] and Myers *et al.*[94] Yet since the focus of our paper is not on studying parallel efficiency, we leave the investigation of parallel efficiency about the dual-grid algorithm in future work.

It should be noted that our framework has the following limitations: (1) it is only applicable to spherical and ellipsoid particles;[58,95] (2) it is still immature when applied to high-Re turbulent flow;[96] and (3) density ratio between particle and fluid should be greater than 1.2.[97]

IV. RESULTS

This section presents several canonical fluid-particle interaction problems to validate the capabilities and robustness of the proposed IAMReX framework. For each case, Δt_0 refers to the time step on level 0, and Δx_0, Δy_0, and Δz_0 are the grid spacings in the x-direction, y-direction, and z-direction, respectively, on level 0.

In our previous work of Zeng *et al.*,[84] we found that with a consistent finest level resolution and fixed refining criteria, more levels of AMR grids can save more computational cost. However, a higher number of levels affects parallel efficiency, especially when different levels have uneven loads. We need to arrange the computational areas for each CPU reasonably to achieve load balance. Within the blocked structured AMR (BSAMR) framework, we typically select an appropriate grid on the coarsest level for all testing cases in this work and then refine it by 2–4 levels to achieve more efficient computation. Regarding the refinement area, since we use BSAMR, we encompass all patches of interest with blocks of a certain size (e.g., size of block = 16, 32, or 64) and provide a 3–5 cell buffer zone for refinement. This ensures that our areas of interest (such as the interior of particles)

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

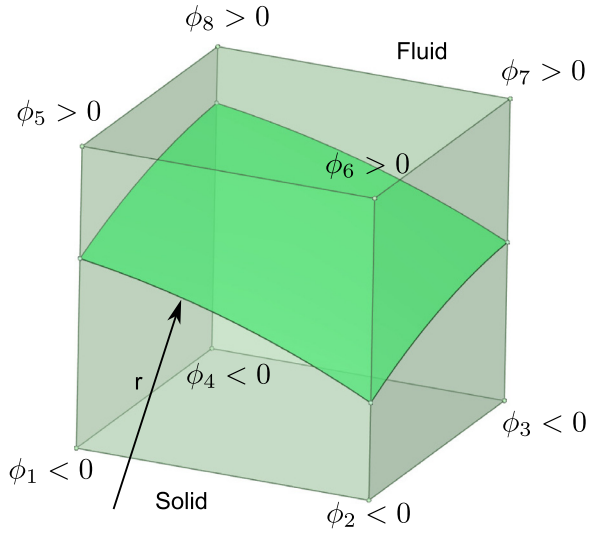


FIG. 6. Sketch of ϕ at each corner of a grid cell with interface.

are always within the refined grid and at a certain distance from the coarse-fine boundary.

A. PVF

The particle volume fraction (PVF), which joins the calculation of free motion updates, is introduced and validated in this section. The PVF is approximated by the signed-distance level-set function ϕ of the fluid-particle interface. The level-set function ϕ is located at cell nodes and is calculated at the eight corners of each cell. The symbol of ϕ as well as the intersected interface is shown in Fig. 6. Here, ϕ is negative inside the particle and positive outside the particle.

Based on the level-set function ϕ, an approximation of PVF can be obtained by the following equation:

$$\alpha_{i,j,k} = \frac{\sum_{m=1}^{8} -\phi_m H(-\phi_m)}{\sum_{m=1}^{8} |\phi_m|}, \tag{16}$$

where H is the Heaviside function, defined by

$$H(\phi) = \begin{cases} 0, & \phi \le 0 \\ 1, & \phi > 0. \end{cases} \tag{17}$$

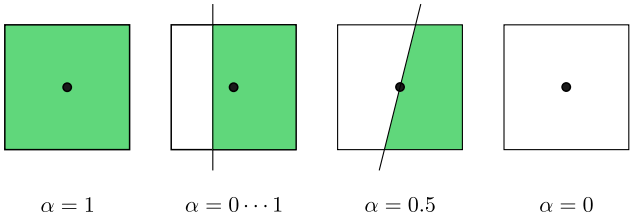


FIG. 7. Sketch of PVF with different values. The white and green colors refer to the cell areas within the fluid and particle, respectively.

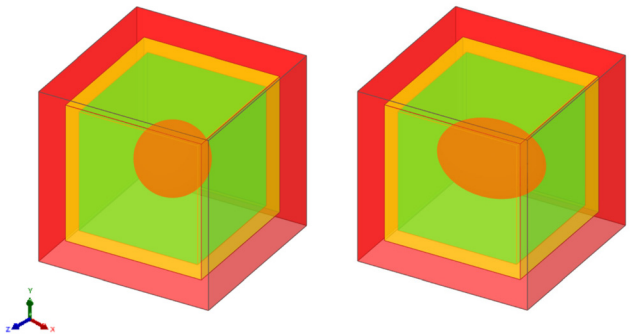


FIG. 8. Results of a three-level AMR grid. The red, orange, and green colors represent the grid on levels 0, 1, and 2, respectively. Left: a spherical particle. Right: an ellipsoidal particle.

On the right-hand side of Eq. (16), the value of ϕ for each cell corner depends on the location of the fluid-particle interface. When the shape of the particle surface is analytically given, the ϕ value can be determined by calculating the Euclidean distance from the corner point of the cell to the particle surface. The calculation of PVF is then transformed from an exact integral to a numerical integral. As shown in Fig. 7, the cell value of PVF varies from 0 to 1, depending on the relative position between the cell center and the fluid-particle interface.

We validate the correctness and convergence of the above PVF approximation by calculating the volumes of spherical and ellipsoidal surfaces on the Cartesian grid. The exact solutions for the sphere and ellipsoid are given by the following formula:

$$V_{exact} = 4\pi \frac{abc}{3}, \tag{18}$$

where a, b, and c are the semi-axes of the ellipsoid. For a sphere case, we have $a = b = c.$, the signed-distance level-set function is as follows:

$$\phi_{i,j,k} = \sqrt{\frac{(x_{i,j,k} - x_p)^2}{a^2} + \frac{(x_{i,j,k} - y_p)^2}{b^2} + \frac{(x_{i,j,k} - z_p)^2}{c^2}} - 1, \tag{19}$$

TABLE I. Calculate sphere volume fraction by using a three-level AMR grid.

d / h	Volume	$\varepsilon_{sphere}[\%]$	s_{sphere}
16	0.266 723 079 6	$5.071 \cdot 10^{-1}$	
32	0.267 763 958 9	$1.188 \cdot 10^{-1}$	2.093 74
64	0.267 999 039 3	$3.116 \cdot 10^{-2}$	1.930 77
128	0.268 062 715 4	$7.407 \cdot 10^{-3}$	2.072 73

TABLE II. Calculate ellipsoid volume fraction by using a three-level AMR grid.

d / h	Volume	$\varepsilon_{ellispoid}[\%]$	$s_{ellispoid}$
16	0.400 490 356 7	$4.062 \cdot 10^{-1}$	
32	0.401 688 497 3	$1.083 \cdot 10^{-1}$	1.907 16
64	0.402 016 644 4	$2.666 \cdot 10^{-2}$	2.022 28
128	0.402 098 289 64	$6.359 \cdot 10^{-3}$	2.0678

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

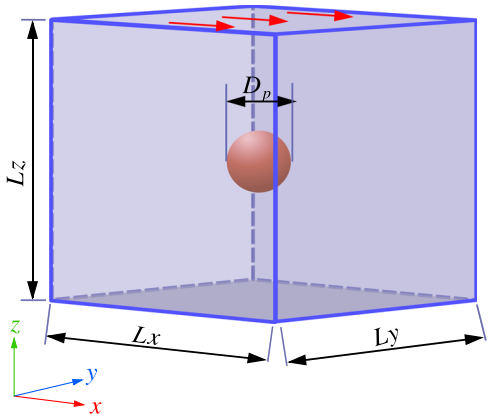


FIG. 9. Schematic of 3D lid-driven cavity flow with fixed spherical particle.

for any Eulerian cell (i, j, k). The computational domain is $L_x \times L_y \times L_z = 2 \times 2 \times 2$, the sphere diameter is $D = 0.8$, and the semi-axes of the ellipsoid are set to be $a = 0.4$, $b = 0.6$, $c = 0.4$. The centers of both two particles are $(x_p, y_p, z_p) = (1, 1, 1)$. As shown in Fig. 8, three levels of AMR grid are used during the PVF calculation, and the particles are enclosed by the finest level.

Tables I and II show the calculation results of the sphere and ellipsoid. The numerical errors decrease with the increase in the d/h, where h is the Cartesian grid spacing on level 0. If the resolution on the finest level remains unchanged, we validated that the results of a three-level grid are the same as those of the corresponding single-level grid. In addition, our results show the second-order convergence and agree well with the results of Kempe and Fröhlich.[40] It also matches the overall second-order accuracy of the basic fluid solver.

Finally, it is noted that this method is also applicable when multiple particles are close to each other or their surfaces are in direct contact. Because the PVF calculation is a separate operation for each particle, the total volume fraction is not needed as long as the Eulerian force considers the effects of all particles.[41]

B. 3D lid-driven cavity with fixed spherical particle

We start to validate the convergence and accuracy of our solver using a lid-driven cavity over a spherical particle case (Fig. 9). The particle's diameter $D_p = 0.25$ and the computation domain $L_x \times L_y \times L_z = 4D_p \times 4D_p \times 4D_p$. The top wall has a constant velocity $U = 1$ at x direction and no-slip stationary conditions are applied on the other wall of the cube and surface of particle. The Reynolds number of the flow is $Re = \rho_f U L_x / \mu_f$. Nine cases are considered in Table III, which includes the single-level, three-level non-subcycling AMR, and three-level subcycling AMRsimulations. Each case also

TABLE III. Parameters of the lid-driven cavity with spherical particle problem.

Case no.	Cells on level 0	Cells on level 1	Cells on level 2	Total cells	l_{max}	Cycling method
1	4 096	⋯	⋯	4 096	0	⋯
2	32 768	⋯	⋯	32 768	0	⋯
3	262 144	⋯	⋯	262 144	0	⋯
4	64	512	4096	4 640	2	None
5	512	4 096	14 336	18 944	2	None
6	406	25 476	114 688	143 360	2	None
7	64	512	4096	4 640	2	Auto
8	512	4 096	14 336	18 944	2	Auto
9	406	25 476	114 688	143 360	2	Auto

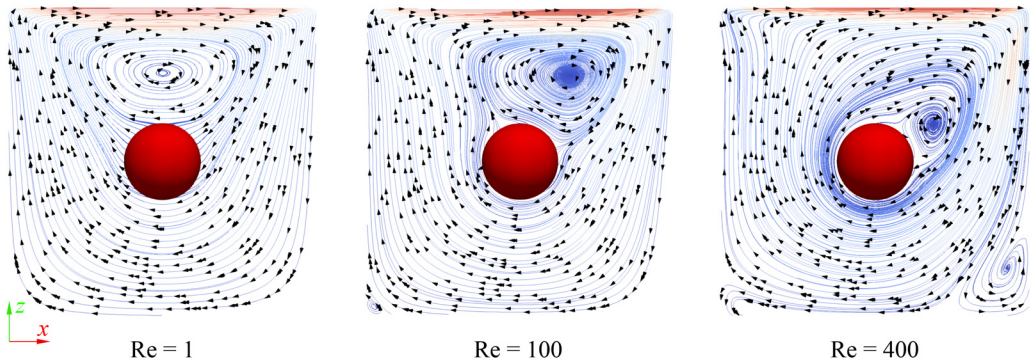


FIG. 10. Streamline contours in the x–z plane at $y = 0.5$ for case 2 in Table III. (a) $Re = 1$; (b) $Re = 100$; (c) $Re = 400$.

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

includes three different Reynolds numbers, i.e., $Re = 1100$ and 400. For the AMR cases, the particle is always refined to the finest level.

We first plot the contour results of case 2 with different Reynolds numbers, which are varied by maintaining a constant flow rate driven by the top wall and changing the viscosity. As shown in Fig. 10, the flow passes around the spherical particle and generates the clockwise vortex. As *Re* increases, the secondary vortex appears near the particle surface and the bottom-right corner.

Figure 11 shows velocity distribution on the x and z direction in the x–z plane at $y = 0.5$. The present results converge as the grid number increases under all three *Re* scenarios. The results of single level, non-subcycling method, and subcycling method produce consistent results, and all of them closely match Young *et al.*[98]

C. Flow past fixed sphere

In this section, we validate the accuracy and efficacy of our adaptive solver by simulating a spherical particle in uniform flow with different particle Reynolds numbers.[52,99] The sketch of the fluid flow passing through a spherical particle is shown in Fig. 12, the diameter

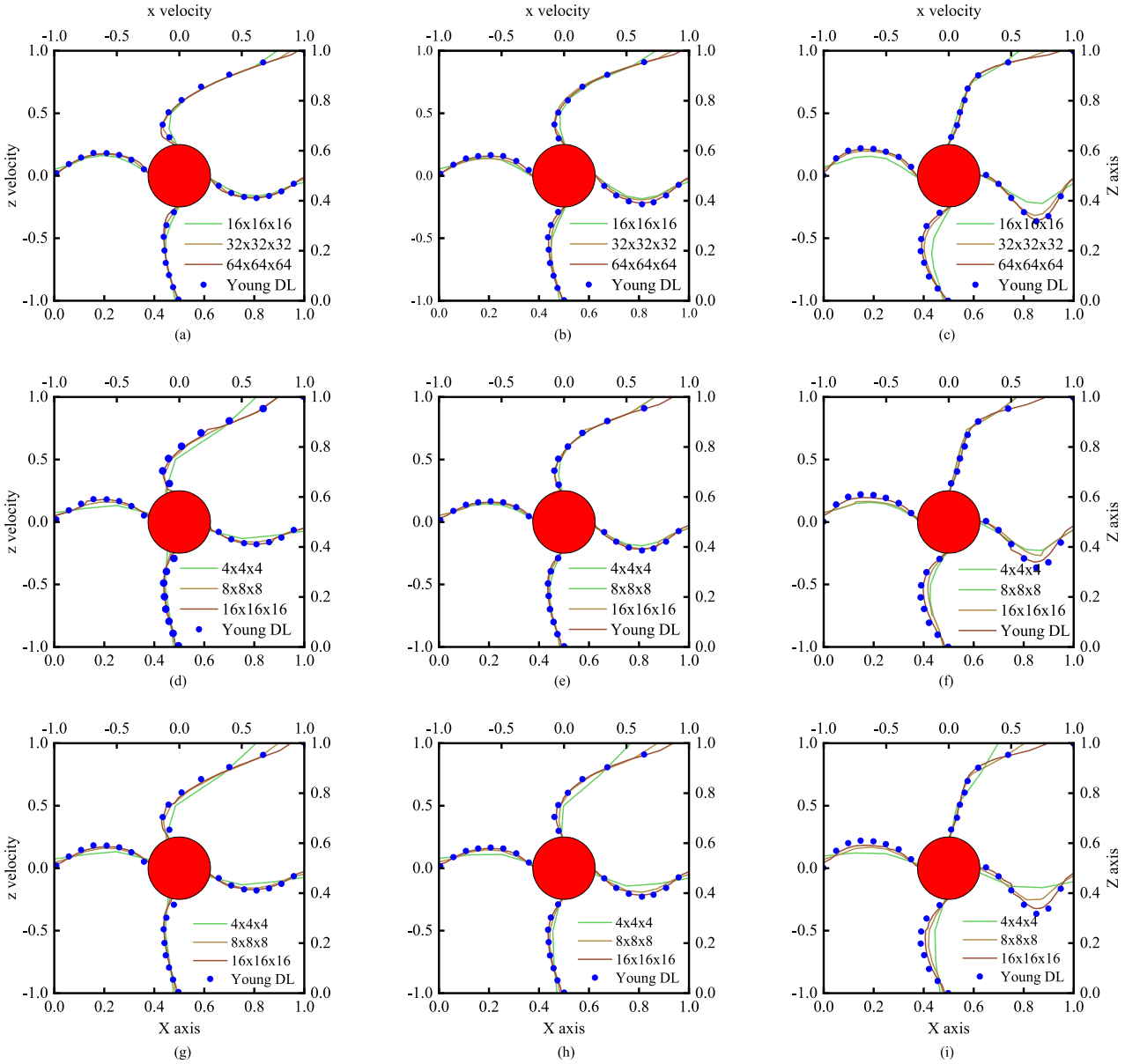


FIG. 11. Velocity profiles at horizontal and vertical line of the lid-driven cavity over a spherical particle problem. Here, (a), (b), and (c) represent the velocity on a single-level grid (cases 1–3) with $Re = 1$, 100, and 400, respectively; (d), (e), and (f) represent $Re = 1$, 100, and 400 of non-subcycling method (cases 4–6) on a three-level AMR grid; (g), (h), and (i) represent $Re = 1$, 100, and 400 of subcycling method (cases 7–9) on a three-level AMR grid.

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

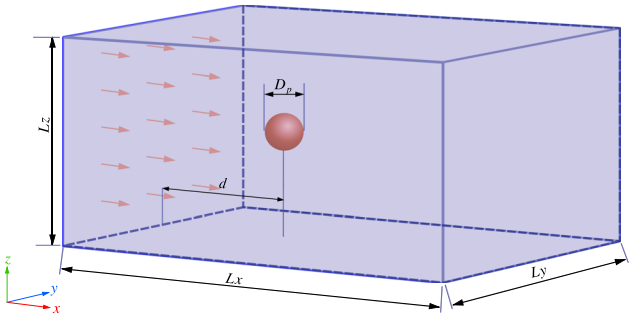


FIG. 12. Schematic of the flow passing through the spherical particles.

of the particle is $D_p = 1$, the computational domain is $L_x \times L_y \times L_z = 20D_p \times 10D_p \times 10D_p$, the distance of the particle from the inlet is $d = 5D_p$ and located in the center of the y–z plane. The inlet and outlet boundaries are applied in the x direction and the inlet velocity U is 1 m/s. Both y and z directions are periodic boundaries.

The influence of AMR on the simulation results is investigated by using the subcycling method with different levels. As shown in Fig. 13, three different types of grid were selected: level 0 indicates the single-level grid containing no AMR, level 1 indicates the two-level grid, and level 2 indicates the three-level grid. For all three types of grid, the ratio of the diameter of the particles to the grid spacing on the finest level is 16. The drag force, including the contributions of Lagrangian force and the PVF function, is calculated by

$$\mathbf{F}_D = -\rho_f \sum_{l=1}^{N_L} \mathbf{F}_l^{n+1/2} \Delta V_l + \rho_f \frac{d}{dt}\left(\int_{V_p} \mathbf{u} dV\right). \tag{20}$$

The theoretical S-N law for calculating the drag coefficient of the shaped particles is as follows:

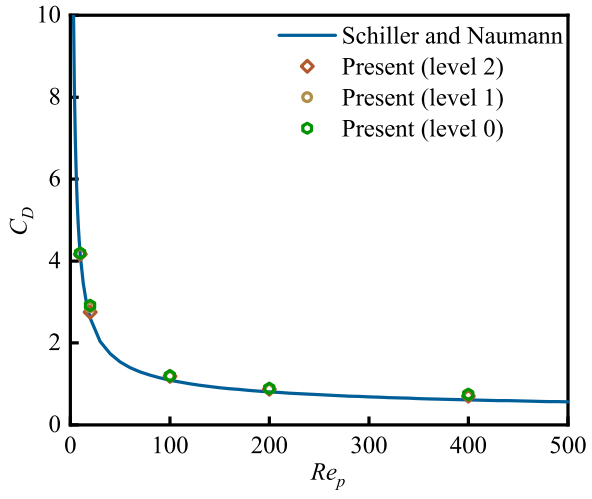


FIG. 13. Drag coefficient of the particle under uniform flows varies with particle Reynolds number at different AMR levels.

$$C_D = (24/Re_p)(1 + 0.15Re_p^{0.687}), \tag{21}$$

which is proposed by Schiller,[99] and $Re_p = UD_p/\nu$ represents the particle Reynolds number. From Fig. 13, it can be seen that the present results under different particle Reynolds numbers are in good agreement with S-N law. The fact that different levels of grid produce the nearly identical results validated the accuracy of our solver on the adaptive grid. Figure 14 illustrates the streamlines of the flow field and vortex form under different Re_p. The streamline is continuous over the coarse-fine boundaries with the help of synchronization operations in Sec. III C 2. As the Reynolds number increases, more unsteadiness appears and the vortex behind the particles gradually becomes asymmetrical as expected.[100]

D. Flow past rotating and moving sphere

The particle is stationary in the previous validation case. In this section, we primarily validate the accuracy and effectiveness of our adaptive solver in handling particles under translational and rotational motion. We consider two different cases: the first case involves a particle with translational degrees of freedom in the x-direction under uniform flow, and the second case involves a particle with rotational degrees of freedom in the z-direction under shear flow.[97] The two cases are shown in Fig. 15(a), where (I) and (II) represent the uniform flow and shear flow, respectively. The size of the computational domain is $L_x \times L_y \times L_z = 30 \times 15 \times 15$, and the particle diameter is $D_p = 1$. The initial position of the particles is at the center of the flow field. The density ratio of particles to the flow field $(R = \rho_p/\rho_f)$ is set as $R = 1.05$ and $R = 5$.[40,97]

For the first case, inlet-outlet boundary conditions were applied in the x direction, while no-slip boundary conditions were used in the y and z directions. For the second case, periodic boundary conditions were applied in the x and z directions. No-slip boundary conditions were applied in the y direction. To simulate different flow field

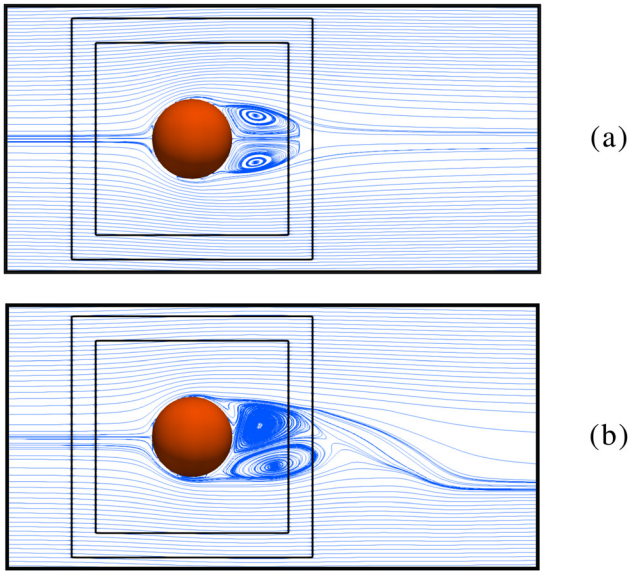


FIG. 14. Streamlines on fluid Eulerian grids around the particle at different particle Reynolds numbers. (a) $Re_p = 100$. (b) $Re_p = 400$.

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

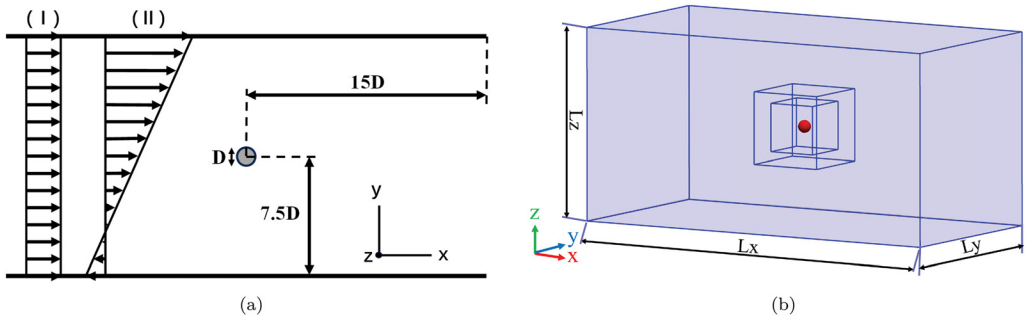


FIG. 15. Flow past rotating and moving sphere example, where (a) is a sketch with (I) representing the uniform flow and (II) representing the shear flow, and (b) is a three-level AMR grid.

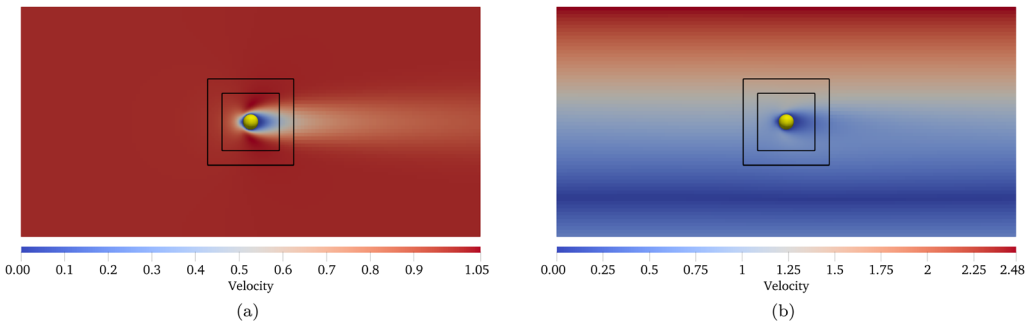


FIG. 16. Steady-state flow field velocity contours in (a) uniform flow and (b) shear flow. The black line represents the finer-level grid.

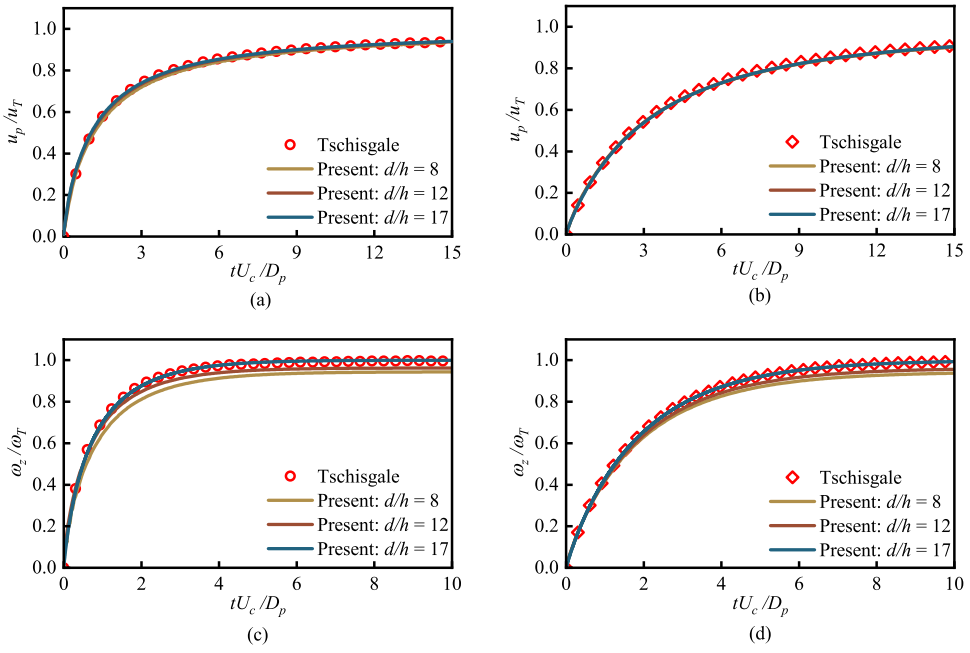


FIG. 17. Comparison of the particle movement between the present results and Tschisgale *et al.*;[97] (a) and (b) show the evolution of the particle's linear velocity over time in uniform flows with density ratios of 1.05 and 5, while (c) and (d) show the evolution of the particle's angular velocity over time in shear flows with density ratios of 1.05 and 5.

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

TABLE IV. Parameters of the falling sphere case with two different density ratios.

ρ_p/ρ_f	$L_z(m)$	$\nu_f(m^2/s)$
2.56	$120D_p$	0.001 042 38
7.71	$180D_p$	0.002 676 26

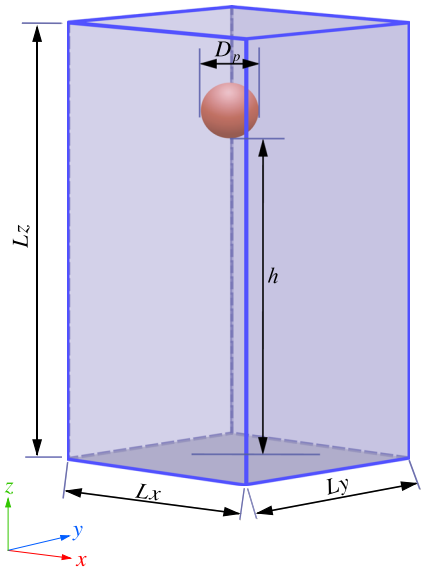


FIG. 18. Sketch of a falling sphere case.

conditions, we adjust the shear rate of the background flow at the inlet by changing the velocities of the upper and lower boundaries. The shear flow is given by

$$\mathbf{u}(y) = U_0 + Sy, \tag{22}$$

where $\mathbf{u}(y)$ represents the velocity at different heights, U_0 represents the velocity of the lower surface, and S is the shear rate. In the uniform flow case, the inlet velocity was set to 1 to simulate the flow field conditions. In the shear flow case, the upper boundary velocity u_{top} is set to be 2.5 and the lower boundary velocity u_{bottom} is set to be −0.5. This velocity setting creates a velocity gradient in the y direction, thus forming a shear effect in the flow field.

In these two cases, a three-level AMR grid is used to save the computational cost without compromising the simulation accuracy. The sketch is shown in Fig. 15(b), the nested cell consists of levels from the outermost to the innermost, designated as levels 0, 1, and 2, respectively. The levels 0, 1, and 2 grid are set to be $N_x \times N_y \times N_z = 128 \times 64 \times 64$, $N_x \times N_y \times N_z = 48 \times 48 \times 48$, and $N_x \times N_y \times N_z = 64 \times 64 \times 64$, respectively. The cell resolution for resolving the particle is measured by the ratio of D_p/h, where D_p represents the particle diameter and h represents the cell spacing on the finest level. We consider three different ratios to validate the convergence of our algorithm in this section, i.e., $D_p/h = 8$, 12, and 17. The definition of the Reynolds number is then defined as

$$Re = \frac{u_p D_p}{\nu}, \tag{23}$$

where u_p is the velocity of the flow field at the height of the center of the particle, and ν is the kinematic viscosity of the flow field. The Reynolds number is chosen as $Re = 20$ for all validation cases in this section.

During the simulation, we first fix the particle and simulate the fluid motion alone. Figures 16(a) and 16(b) show the steady flow field under the uniform inflow and the shear inflow, respectively. Flow field is continuous and smooth across the coarse-fine boundaries with the help of synchronization operations in Sec. III C 2. Once the fluid reaches a steady state, we release the particle and record the variation of the particle's translational velocity in the x-direction over time under uniform flow, as well as the variation of the particle's angular velocity in the z-direction over time under shear flow. As shown in Fig. 17, increasing D_p/h achieves better convergence. The results of $D_p/h = 17$ are in good agreement with data in Tschisgale *et al.*,[97] which demonstrates the accuracy of our adaptive solver in dealing with particle motions with different degrees of freedom.

E. Falling sphere

The falling sphere example is used to compare the subcycling and non-subcycling methods for particles with free motions.[38,40] We start with the computational setup Uhlmann[38] and consider two particle

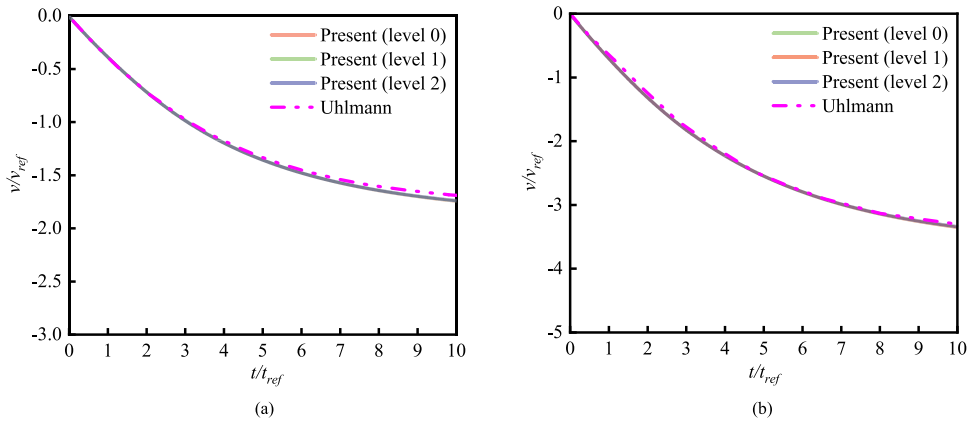


FIG. 19. Comparison of the settling velocity for different density ratios between the present results and Uhlmann:[38] (a) $\rho_p/\rho_f = 2.56$; (b) $\rho_p/\rho_f = 7.71$.

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

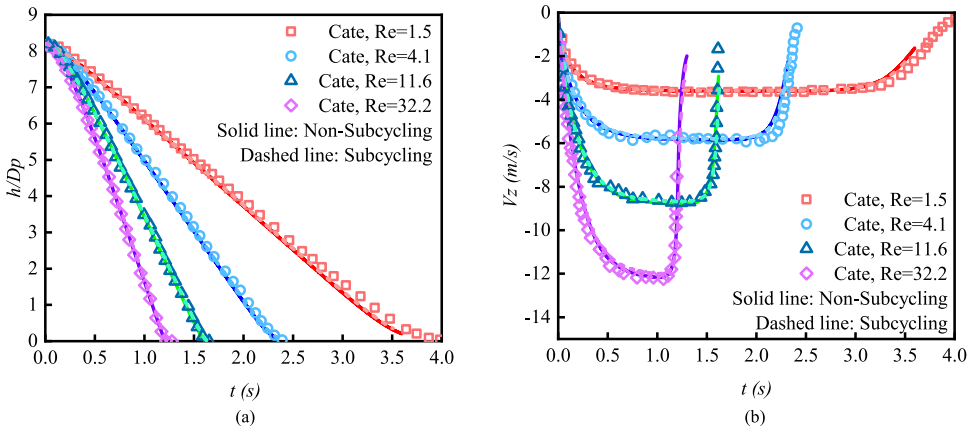


FIG. 20. Comparison of the subcycling and non-subcycling results with Ten Cate *et al.*:[101] (a) the time series of the dimensionless height; (b) the time series of the dimensional z-velocity.

density ratios ρ_p/ρ_f in Table IV. As the sketch in Fig. 18, the computational domain is $L_x \times L_y \times L_z = 15D_p \times 15D_p \times L_z$, in which the particle diameter is $D_p = 1/6$. The initial position of the particles is $(x, y, z) = (12.5D_p, 12.5D_p, 114D_p)$, and the gravitational acceleration $g = -9.8\,\mathrm{m/s}$ is in the z-axis. For all of following cases, the ratio of the particle diameter to the spacing of the finest grid is set to be 27.

Figure 19 shows the present results on different levels using the non-subcycling method, in which $u_{ref} = \sqrt{|g|D_p}$ and $t_{ref} = \sqrt{D_p/|g|}$ are the reference velocity and time, respectively. The present results validate the consistency of our solver on different levels and show good agreement with Uhlmann[38] on both high-density ratio (i.e., $\rho_p/\rho_f = 7.71$) and relatively low-density ratio (i.e., $\rho_p/\rho_f = 2.56$).

We further test our solver on another canonical setup.[101] The size of the entire calculation domain is $Lx \times Ly \times L_z = 6.67D_p \times 6.67D_p \times 13.34D_p$, the diameter of the particle $Dp = 0.015$, and the initial position of the particle is $(x, y, z) = (3.33D_p, 3.33D_p, 8.7D_p)$. The particle is released from the initial position until it touches the bottom boundary. The gravity $g = -9.81\,\mathrm{m/s^2}$ is vertically downward and the density of the particles is $\rho_p = 1120\,\mathrm{kg/m^3}$. For the bottom and side boundaries, both of them are no-slip boundaries, while the top is a free-slip boundary. The grid number on level 0 is $32 \times 32 \times 64$. Other parameters are listed in Fig. 5, in which four cases with AMR are set up and each of them utilizes both the subcycling and non-subcycling methods.

TABLE V. Parameters of the same falling sphere case as Ten Cate *et al.*[101]

Case no.	$\rho_f(kg/m^3)$	$\nu_f(m^2/s)$	Re	l_{max}
1	970	3.845×10^{-4}	1.5	2
2	965	2.197×10^{-4}	4.1	2
3	962	1.175×10^{-4}	11.6	2
4	960	6.042×10^{-5}	32.2	2

As shown in Fig. 20, our present numerical results achieve good agreement with the experimental results of Ten Cate *et al.*[101] For cases with different density ratios, the particle accelerates at the initial stage, reaches the steady state, and then touches the wall. The overlapping

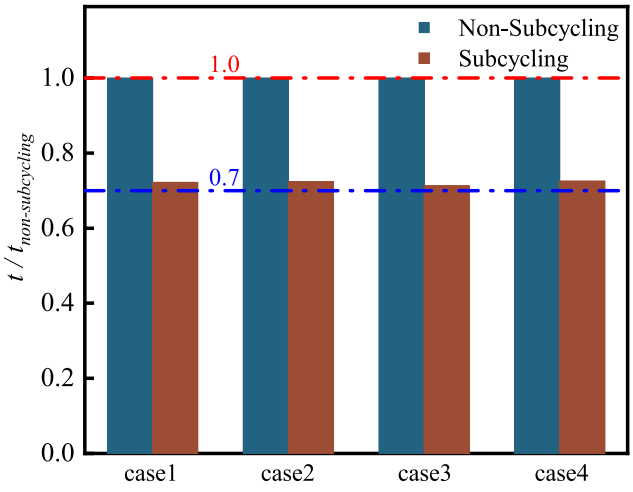


FIG. 21. Comparison of the computational cost between the subcycling and non-subcycling method for all falling sphere cases in Table V.

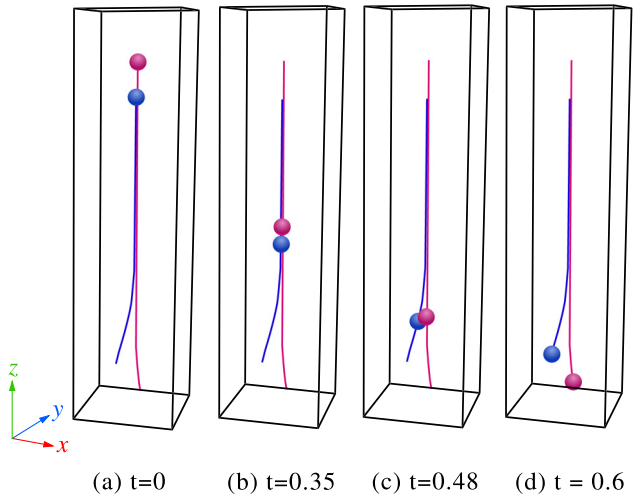


FIG. 22. Trajectory of two particles and their positions at different time steps.

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

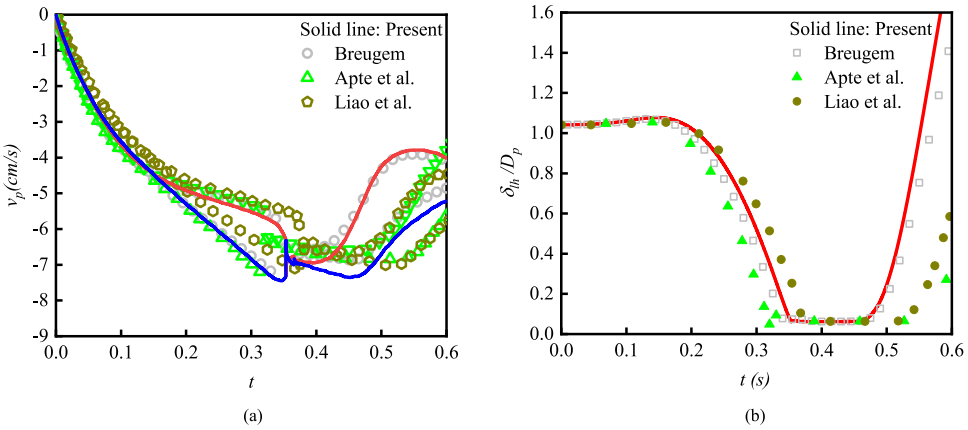


FIG. 23. (a) Time series of the vertical velocities of the two particles. (b) Time series of the distance of two particles. The present results are compared with Breugem,[41] Apte *et al.*,[102] and Liao *et al.*[103]

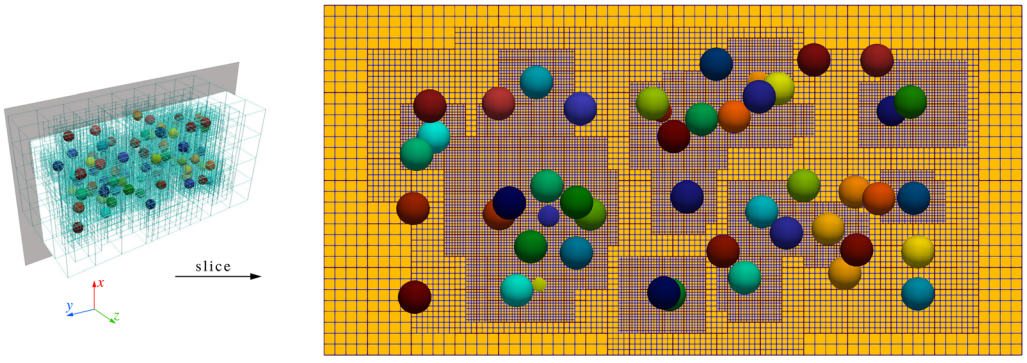


FIG. 24. Monodisperse particles on a three-level AMR grid.

between the solid line and the dashed line also validates the accuracy and consistency of our subcyling and non-subcycling methods for particles with free motions. In Fig. 21, the computational cost between the subcycling method is compared with the non-subcycling method for all four cases in Table V. During the comparison, the running time, without including the IO process, is added and normalized by the total time of the non-subcycling method. It is seen that the subcycling method is more efficient in the falling sphere simulation, which takes around 30 percent less time compared with the non-subcycling time.

F. Drafting-kissing-tumbling

In this section, we study the drafting, kissing, and tumbling (DKT) phenomenon of a pair of particles. This phenomenon is frequently observed in particle sedimentations and has been studied in previous work.[38,52,100] The objective here is to validate the correctness and accuracy of our adaptive solver when a collision model of two particles is combined with AMR. Following the previous work of Breugem,[41] a collision model between the particles is added as follows:

$$F_{c,ij} = \begin{cases} -\dfrac{m_p||\mathbf{g}||}{\varepsilon_c}\left(\dfrac{||\delta_{i,j}|| - D_c}{d_c}\right)^2 \dfrac{\delta_{i,j}}{||\delta_{i,j}||}, & |\delta_{i,j}| < D_c \\ 0, & |\delta_{i,j}| > D_c. \end{cases} \quad (24)$$

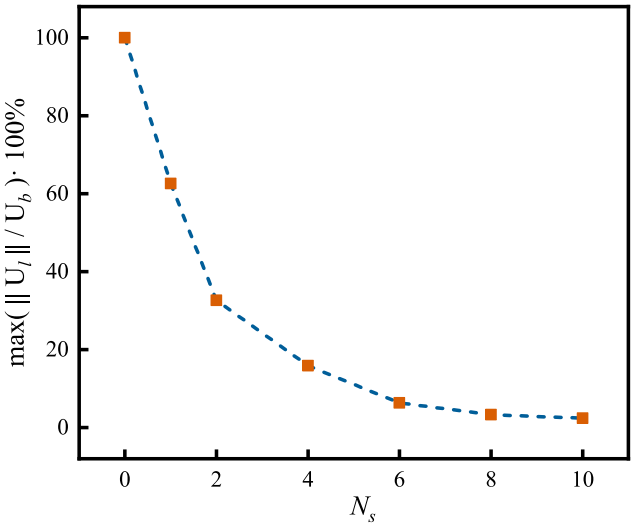


FIG. 25. Maximum error in the norm of the Lagrangian velocity $||U_l||$ as function of the number of iterations N_s. $||U_l||$ is nondimensionalized by the bulk velocity of the unit flow U_b.

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

TABLE VI. Number of grid cells for monodisperse particle cases.

Case no.	Level 0 cells	Level 1 cells	Level 2 cells	Total cells
1	8 192 000	⋯	⋯	8 192 000
2	128 000	534 656	1 593 664	2 256 320

In Eq. (24), F_c refers to the repulsive force, m denotes the mass of the particles, $\mathbf{g}$ represents the gravitational acceleration, δ represents the distance between the two particles, and D_c represents the sum of the radius of the two particles and the grid size. The d_c represents the grid spacing on the finest level and the dimensionless constant ε_c is set to be 10^{-4}.

In this DKT case, the diameters of the two particles are the same, $D_p = 1.67mm$, and the computational domain is $L_x \times L_y \times L_z = 6D_p \times 6D_p \times 24D_p$. The initial position of the particle at the higher location is $(x_h, y_h, z_h) = (3.03D_p, 3.03D_p, 21.0D_p)$, while the particle at the lower location starts at $(x_l, y_l, z_l) = (2.97D_p, 2.97D_p, 18.96D_p)$. In addition, the physical parameters of the fluid flow and particles are as follows: $\rho_p = 1140\,\mathrm{kg/m^3}$, $\rho_f = 1000\,\mathrm{kg/m^3}$, and $\nu_f = 10^{-6}\,\mathrm{m^2/s}$. The gravity $g = -9.81\,\mathrm{m/s^2}$ is vertically downward. All boundaries of the computational domain have no-slip conditions, and the ratio of the particle diameter to the grid spacing on the finest level is consistent in all directions, i.e., $D_p/d_c = 16$.

Figure 22 shows the motion trajectories of the particles and their positions at different time steps. At the initial stage of the sedimentation process, the upper particle is slightly higher than the lower particle. From $t = 0$ to $t = 0.35$, the lower pressure in the wake of the lower particle results in the larger velocity of the upper particle, which helps it to gradually catch up with the lower particle. This process is known as "drafting." Then, the distance between the two particles gradually decreases until they eventually collide. From $t = 0.35$ to $t = 0.48$ in Figs. 22(b) and 22(c), the distance between the two particles remains almost constant, and during this period, the upper particle gradually shifts to the side of the lower particle. This process is called "kissing." After the "kissing" stage, the particle that was originally higher flips to the side of the lower particle, and the unstable vertical alignment makes the upper particle push the lower one aside and take the lead. This process is called "tumbling." Figure 22 shows that our present results can qualitatively and reasonably reproduce the DKT process.[41,100]

The time series of the distance and the vertical velocities of these two particles are shown in Fig. 23. Our adaptive results with the non-subcycling method agree well with the data of Breugem,[41] Apte *et al.*,[102] and Liao *et al.*[103]

G. Cluster of monodisperse particles

As the last example, we demonstrate the accuracy and efficacy of our codes for simulating clusters of particles on the multi-level grid. As shown in Fig. 24, 80 particles of diameter D = 1 are randomly distributed in a channel of size $L_x \times L_y \times L_z = 10 \times 20 \times 10$. To choose an optimal interaction number N_s in this complex configuration, the maximum error of the no-slip boundary condition among 80 particles is tested with a unit flow field $\boldsymbol{u} = (1, 0, 0)$. As shown in Fig. 25, the maximum error of no-slip condition decreases as N_s increases and it is strongly reduced for $N_s = 2$. According to the selection suggestions provided by Breugem,[41] $N_s = 2$ is the optimal value for balancing the accuracy of the no-slip boundary and the computational efficiency. After determining N_s, the fluid flow is driven by applying a pressure gradient of 1.0 in the z direction. This case can represent a porous medium with a volume fraction of 0.02. Three levels of the AMR grid is applied. The grid resolution on the finest level is $d/h = 16$. As shown in Table VI, the total number of cells in the AMR grid is 2 256 320. Since the multi-direct forcing immersed boundary method[49,104] and fictitious domain method[105,106] require cube grid cells, the grid cell requirement is equals to case 1 in Table VI. Compared with them, our algorithm has a 72.5% grid reduction with $d/h = 16$. It has a 62.5% Lagrangian markers reduction compared with the DLM method[74,107] with $d/h = 16$.

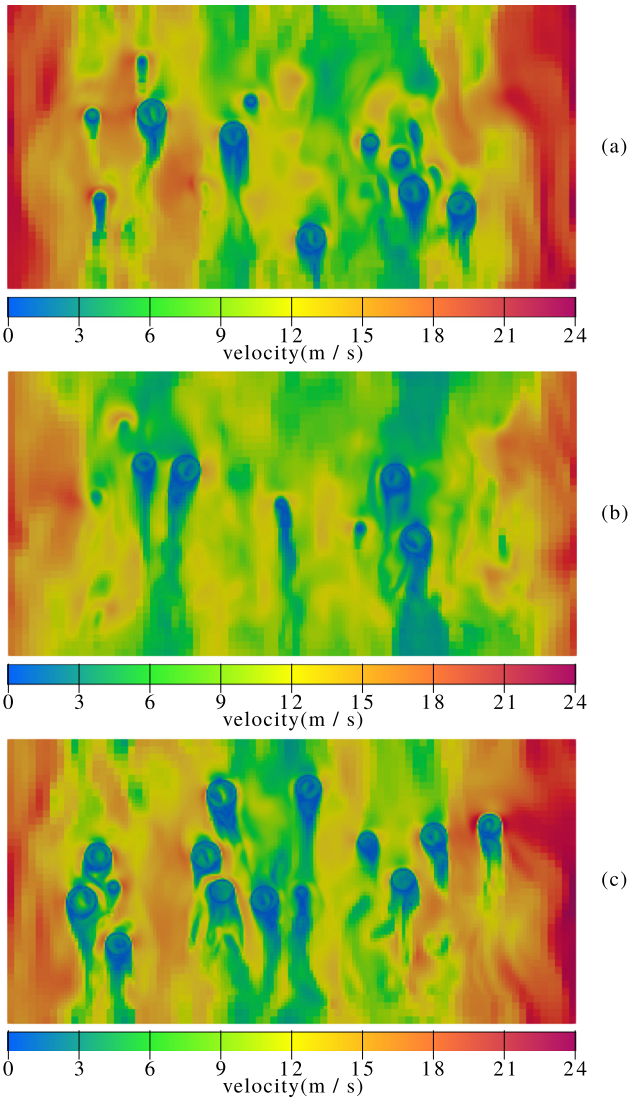


FIG. 26. Contours of velocity magnitude in y–z plane at the different x position: (a) $x = 2.5$; (b) $x = 5.0$; (c) $x = 7.5$.

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

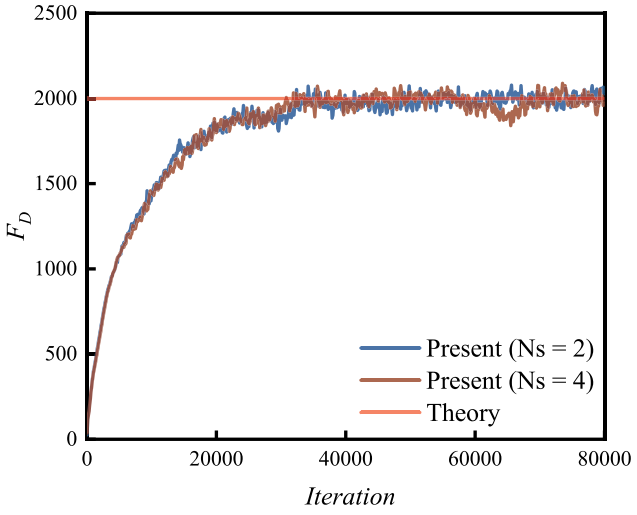


FIG. 27. Comparison of total drag force between the theory and present results with different N_s.

When the simulation reaches the steady state, the total pressure drop balances the IB force generated by all particles in the streamwise z direction. Following the equation in Ref. 57, the theoretical drag force would be as follows:

$$F_{theory} = \left(\frac{\Delta p}{\Delta z} L_z\right) L_x L_y. \tag{25}$$

Figure 26 represents the velocity contour of three different interfaces in the x direction at the steady state. The flow passes around the particles and generates the wakes behind. Figure 27 shows the time series of total IB force for all particles. The resistance gradually reaches a steady state after 40 000 steps. In this case, the theoretical value of drag force given by Eq. (25) is 2000, while the present average values at steady state with $N_s = 2$ and 4 are all converged around 2000. It indicates that $N_s = 2$ is sufficient for this case. The agreement between theory and present results validates the accuracy of our proposed framework in dealing with large amounts of particles in the fluid system.

V. CONCLUSION

In this work, we established a novel adaptive solver with subcycling and non-subcycling time advancement methods for simulating fluid-particle interaction. The proposed multi-level advancement algorithm uses the level-by-level advancement technique for time-marching the variables in the adaptive meshes and decouples the time advancement at different levels. When the subcycling method is applied, the time step constraint on the coarser levels is relaxed compared to the finer levels. The accuracy and efficacy of both subcycling and non-subcycling methods are validated by the classic flow past sphere and falling sphere cases.

We also implemented different types of constraints for particles, including the prescribed and free motions. The numerical approximations of the PVF variable are shown to be second-order accurate and match our time advancement algorithms.[84] The particle motions are also validated using the sphere in uniform and shear flow cases. In addition, the collision model with repulsive force is correctly added to account for particle–particle collision and validated by the DKT case on the adaptive meshes. More advanced collision models, such as the solid sphere model (SSM) and adaptive collision time model (ACTM),[100,108] will be included in future work.

We brought two memory optimization techniques of our adaptive solver. First, the Lagrangian markers associated with particles only exist on the finest level of the adaptive grid. Since the Eulerian–Lagrangian information exchange information only happens on the finest level, we do not need to define any auxiliary variables and particle information on the coarser levels. This helps to save memory compared with our previous study,[74] in which coarser levels also store Eulerian forces for using the force averaging schemes. Second, there is only one set of Lagrangian markers when many particles are simulated. Since we loop over particles, memory associated with Lagrangian markers can be reused.

At last, the cluster of monodispersed particles case shows the accuracy and robustness of the computational framework while simulating large amounts of particles. The three-level AMR simulation leads to a 72.46% grid reduction compared with the particle resolved simulation without AMR. This capability enabled us to simulate a bunch of practical engineering problems, including eolian sand and dust movement, sediment transport, and fluidized-bed processes. The codes are openly available on GitHub and the raw postprocessing data are also attached for reproducing all results in Sec. IV.

Regarding ongoing work, we will extend our adaptive solver and add the new features, including GPU running,[5,91] fully implicit schemes of fluid-particle coupling,[109] non-Newtonain fluids,[83,110] and soft-sphere discrete element method (DEM).[52,100,111] Considering different shapes of particles[112] and running the simulations on the non-uniform grid[56,57] are also two promising directions for practical applications.

ACKNOWLEDGMENTS

X.L., Y.Z., and Z.Z. are grateful to Ann Almgren, Andy Nonaka, Andrew Myers, Axel Huebl, and Weiqun Zhang in the Lawrence Berkeley National Laboratory (LBNL) for their discussions related to AMReX and IAMR. Y.Z. and Z.Z. also thank Professor Lian Shen, Professor Ruifeng Hu, and Professor Xiaojing Zheng for their help during Ph.D. studies.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Xuzhu Li: Conceptualization (equal); Data curation (equal); Investigation (equal); Methodology (equal); Software (equal); Validation (equal); Visualization (equal); Writing – original draft (equal). **Chun Li:** Data curation (equal); Formal analysis (equal); Investigation (equal); Validation (equal); Writing – original draft (equal). **Xiaokai Li:** Data curation (equal); Formal analysis (equal); Validation (equal); Writing – original draft (equal). **Wenzhuo Li:** Data curation (equal); Investigation (equal); Validation (equal); Writing – original draft (equal). **Mingze Tang:** Data curation (equal);

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

Investigation (equal); Validation (equal); Writing – original draft (equal). **Yadong Zeng:** Conceptualization (equal); Data curation (equal); Investigation (equal); Methodology (equal); Software (equal); Supervision (equal); Validation (equal); Visualization (equal); Writing – original draft (equal); Writing – review & editing (equal). **Zhengping Zhu:** Conceptualization (equal); Data curation (equal); Investigation (equal); Methodology (equal); Resources (equal); Supervision (equal); Validation (equal); Writing – original draft (equal); Writing – review & editing (equal).

DATA AVAILABILITY

The codes that support the findings of this study are openly available in https://github.com/ruohai0925/IAMR/tree/development. Please help to submit a pulling request (PR) on GitHub if needed. Raw postprocessing data of all cases can be downloaded from https://pan.baidu.com/s/1bZRoDunjBv7bqYL8CI3ASA?pwd=i5c2. The gallery about the cluster of monodisperse particles can be accessed at https://amrex-codes.github.io/amrex/gallery.html.

REFERENCES

[1]S. Balachandar and J. K. Eaton, "Turbulent dispersed multiphase flow," Annu. Rev. Fluid Mech. **42**, 111–133 (2010).
[2]L. Brandt and F. Coletti, "Particle-laden turbulence: Progress and perspectives," Annu. Rev. Fluid Mech. **54**, 159–189 (2022).
[3]Y. Liu, X. Liu, Z. Zhang, J. Lu, Y. Wang, K. Xu, H. Zhu, B. Wang, L. Lin, and W. Xue, "Experimental and fluid flow simulation studies of laser-electrochemical hybrid manufacturing of micro–nano symbiotic superamphiphobic surfaces," J. Chem. Phys. **159**, 114702 (2023).
[4]B. Niu, M. Ahmed, T. Wen, G. Xu, and X. Liu, "Particle-resolved CFD modeling of the flow and heat transfer characteristics of the fluid in a fixed bed," Powder Technol. **421**, 118445 (2023).
[5]D. Liu, S. He, H. Cheng, and Y. Zeng, "Investigate the efficiency of incompressible flow simulations on CPUs and GPUs with BSAMR," arXiv:2405.07148 (2024).
[6]W. Sadowski, M. Sayyari, F. di Mare, C. Velten, and K. Zähringer, "Particle-resolved simulations and measurements of the flow through a uniform packed bed," Phys. Fluids **36**, 023330 (2024).
[7]A. A. Zaidi, "Particle resolved direct numerical simulations for heat transfer analysis of a spouted bed," Powder Technol. **433**, 119239 (2024).
[8]M. Mazzuoli, P. Blondeaux, and G. Vittori, "Particle-resolved direct numerical simulation of the oscillatory flow and sediment motion over a rippled bed," Int. J. Multiphase Flow **172**, 104707 (2024).
[9]K. D. Squires and J. K. Eaton, "Particle response and turbulence modification in isotropic turbulence," Phys. Fluids A: Fluid Dyn. **2**, 1191–1203 (1990).
[10]L.-P. Wang and M. R. Maxey, "Settling velocity and concentration distribution of heavy particles in homogeneous isotropic turbulence," J. Fluid Mech. **256**, 27–68 (1993).
[11]A. Ferrante and S. Elghobashi, "On the physical mechanisms of two-way coupling in particle-laden isotropic turbulence," Phys. fluids **15**, 315–329 (2003).
[12]M. W. Vance, K. D. Squires, and O. Simonin, "Properties of the particle velocity field in gas-solid turbulent channel flow," Phys. Fluids **18**, 063302 (2006).
[13]L. Zhao, H. I. Andersson, and J. Gillissen, "Turbulence modulation and drag reduction by spherical particles," Phys. Fluids **22**, 081702 (2010).
[14]J. Lee and C. Lee, "Modification of particle-laden near-wall turbulence: Effect of stokes number," Phys. Fluids **27**, 023303 (2015).
[15]D. Li, K. Luo, and J. Fan, "Modulation of turbulence by dispersed solid particles in a spatially developing flat-plate boundary layer," J. Fluid Mech. **802**, 359–394 (2016).
[16]G. Wang and D. Richter, "Two mechanisms of modulation of very-large-scale motions by inertial particles in open channel flow," J. Fluid Mech. **868**, 538–559 (2019).
[17]X. Zheng, S. Feng, and P. Wang, "Modulation of turbulence by saltating particles on erodible bed surface," J. Fluid Mech. **918**, A16 (2021).
[18]Y. Pan and S. Banerjee, "Numerical investigation of the effects of large particles on wall-turbulence," Phys. Fluids **9**, 3786–3807 (1997).
[19]P. Bagchi and S. Balachandar, "Effect of turbulence on the drag and lift of a particle," Phys. Fluids **15**, 3496–3513 (2003).
[20]T. M. Burton and J. K. Eaton, "Fully resolved simulations of particle-turbulence interaction," J. Fluid Mech. **545**, 67–111 (2005).
[21]X. Shao, T. Wu, and Z. Yu, "Fully resolved numerical simulation of particle-laden turbulent flow in a horizontal channel at a low reynolds number," J. Fluid Mech. **693**, 319–344 (2012).
[22]F. Picano, W.-P. Breugem, and L. Brandt, "Turbulent channel flow of dense suspensions of neutrally buoyant spheres," J. Fluid Mech. **764**, 463–487 (2015).
[23]L.-P. Wang, C. Peng, Z. Guo, and Z. Yu, "Flow modulation by finite-size neutrally buoyant particles in a turbulent channel flow," J. Fluids Eng. **138**, 041306 (2016).
[24]Y. Wang, Z. Zhu, R. Hu, and L. Shen, "Direct numerical simulation of a stationary spherical particle in fluctuating inflows," AIP Adv. **12**, 025019 (2022).
[25]P. Wang, Y. Lei, Z. Zhu, and X. Zheng, "Drag model of finite-sized particle in turbulent wall-bound flow over sediment bed," J. Fluid Mech. **964**, A9 (2023).
[26]R. Mittal and G. Iaccarino, "Immersed boundary methods," Annu. Rev. Fluid Mech. **37**, 239–261 (2005).
[27]F. Sotiropoulos and X. Yang, "Immersed boundary methods for simulating fluid–structure interaction," Prog. Aerosp. Sci. **65**, 1–21 (2014).
[28]B. E. Griffith and N. A. Patankar, "Immersed methods for fluid–structure interaction," Annu. Rev. Fluid Mech. **52**, 421–448 (2020).
[29]R. Verzicco, "Immersed boundary methods: Historical perspective and future outlook," Annu. Rev. Fluid Mech. **55**, 129–155 (2023).
[30]D. Goldstein, R. Handler, and L. Sirovich, "Modeling a no-slip flow boundary with an external force field," J. Comput. Phys. **105**, 354–366 (1993).
[31]E. M. Saiki and S. Biringen, "Numerical simulation of a cylinder in uniform flow: Application of a virtual boundary method," J. Comput. Phys. **123**, 450–465 (1996).
[32]P. Angot, C.-H. Bruneau, and P. Fabrie, "A penalization method to take into account obstacles in incompressible viscous flows," Numerische Math. **81**, 497–520 (1999).
[33]M. Specklin and Y. Delauré, "A sharp immersed boundary method based on penalization and its application to moving boundaries and turbulent rotating flows," Eur. J. Mech.-B/Fluids **70**, 130–147 (2018).
[34]M.-C. Lai and C. S. Peskin, "An immersed boundary method with formal second-order accuracy and reduced numerical viscosity," J. Comput. Phys. **160**, 705–719 (2000).
[35]C. Lee, "Stability characteristics of the virtual boundary method in three-dimensional applications," J. Comput. Phys. **184**, 559–591 (2003).
[36]E. A. Fadlun, R. Verzicco, P. Orlandi, and J. Mohd-Yusof, "Combined immersed-boundary finite-difference methods for three-dimensional complex flow simulations," J. Comput. Phys. **161**, 35–60 (2000).
[37]B. E. Griffith and C. S. Peskin, "On the order of accuracy of the immersed boundary method: Higher order convergence rates for sufficiently smooth problems," J. Comput. Phys. **208**, 75–105 (2005).
[38]M. Uhlmann, "An immersed boundary method with direct forcing for the simulation of particulate flows," J. Comput. Phys. **209**, 448–476 (2005).
[39]K. Luo, Z. Wang, J. Fan, and K. Cen, "Full-scale solutions to particle-laden flows: Multidirect forcing and immersed boundary method," Phys. Rev. E **76**, 066709 (2007).
[40]T. Kempe and J. Fröhlich, "An improved immersed boundary method with direct forcing for the simulation of particle laden flows," J. Comput. Phys. **231**, 3663–3684 (2012).
[41]W.-P. Breugem, "A second-order accurate immersed boundary method for fully resolved simulations of particle-laden flows," J. Comput. Phys. **231**, 4469–4498 (2012).
[42]D. F. Martin and P. Colella, "A cell-centered adaptive projection method for the incompressible Euler equations," J. Comput. Phys. **163**, 271–312 (2000).
[43]D. F. Martin, P. Colella, and D. Graves, "A cell-centered adaptive projection method for the incompressible Navier–Stokes equations in three dimensions," J. Comput. Phys. **227**, 1863–1886 (2008).

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

[44]A. S. Almgren, J. B. Bell, P. Colella, L. H. Howell, and M. L. Welcome, "A conservative adaptive projection method for the variable density incompressible Navier–Stokes equations," J. Comput. Phys. **142**, 1–46 (1998).
[45]C. Ji, A. Munjiza, E. Avital, J. Ma, and J. Williams, "Direct numerical simulation of sediment entrainment in turbulent channel flow," Phys. Fluids **25**, 056601 (2013).
[46]C. Ji, A. Munjiza, E. Avital, D. Xu, and J. Williams, "Saltation of particles in turbulent channel flow," Phys. Rev. E **89**, 052202 (2014).
[47]A. G. Kidanemariam and M. Uhlmann, "Direct numerical simulation of pattern formation in subaqueous sediment," J. Fluid Mech. **750**, R2 (2014).
[48]A. G. Kidanemariam and M. Uhlmann, "Formation of sediment patterns in channel flow: Minimal unstable systems and their temporal evolution," J. Fluid Mech. **818**, 716–743 (2017).
[49]A. G. Kidanemariam, M. Scherer, and M. Uhlmann, "Open-channel flow over evolving subaqueous ripples," J. Fluid Mech. **937**, A26 (2022).
[50]M. Scherer, M. Uhlmann, A. G. Kidanemariam, and M. Krayer, "On the role of turbulent large-scale streaks in generating sediment ridges," J. Fluid Mech. **930**, A11 (2022).
[51]B. Vowinckel, R. Jain, T. Kempe, and J. Fröhlich, "Entrainment of single particles in a turbulent open-channel flow: A numerical study," J. Hydraulic Res. **54**, 158–171 (2016).
[52]Z. Zhu, R. Hu, Y. Lei, L. Shen, and X. Zheng, "Particle resolved simulation of sediment transport by a hybrid parallel approach," Int. J. Multiphase Flow **152**, 104072 (2022).
[53]R. Jain, S. Tschisgale, and J. Froehlich, "Effect of particle shape on bedload sediment transport in case of small particle loading," Meccanica **55**, 299–315 (2020).
[54]A. M. Roma, C. S. Peskin, and M. J. Berger, "An adaptive version of the immersed boundary method," J. Comput. Phys. **153**, 509–534 (1999).
[55]W. K. Liu, S. Jun, and Y. F. Zhang, "Reproducing kernel particle methods," Numer. Methods Fluids **20**, 1081–1106 (1995).
[56]A. Pinelli, I. Naqavi, U. Piomelli, and J. Favier, "Immersed-boundary methods for general finite-difference and finite-volume Navier–Stokes solvers," J. Comput. Phys. **229**, 9073–9091 (2010).
[57]G. Akiki and S. Balachandar, "Immersed boundary method with non-uniform distribution of Lagrangian markers for a non-uniform Eulerian mesh," J. Comput. Phys. **307**, 34–59 (2016).
[58]J. Jang and C. Lee, "An immersed boundary method for nonuniform grids," J. Comput. Phys. **341**, 1–12 (2017).
[59]M. J. Berger and J. Oliger, "Adaptive mesh refinement for hyperbolic partial differential equations," J. Comput. Phys. **53**, 484–512 (1984).
[60]M. J. Berger and P. Colella, "Local adaptive mesh refinement for shock hydrodynamics," J. Comput. Phys. **82**, 64–84 (1989).
[61]A. Guittet, M. Theillard, and F. Gibou, "A stable projection method for the incompressible Navier–Stokes equations on arbitrary geometries and adaptive Quad/Octrees," J. Comput. Phys. **292**, 215–238 (2015).
[62]M. Mirzadeh, A. Guittet, C. Burstedde, and F. Gibou, "Parallel level-set methods on adaptive tree-based grids," J. Comput. Phys. **322**, 345–364 (2016).
[63]S. Popinet, "Gerris: A tree-based adaptive solver for the incompressible Euler equations in complex geometries," J. Comput. Phys. **190**, 572–600 (2003).
[64]C. Burstedde, L. C. Wilcox, and O. Ghattas, "p4est: Scalable algorithms for parallel adaptive mesh refinement on forests of octrees," SIAM J. Sci. Comput. **33**, 1103–1133 (2011).
[65]B. S. Kirk, J. W. Peterson, R. H. Stogner, and G. F. Carey, "libMesh: A c++ library for parallel adaptive mesh refinement/coarsening simulations," Eng. Comput. **22**, 237–254 (2006).
[66]B. Fryxell, K. Olson, P. Ricker, F. X. Timmes, M. Zingale, D. Lamb, P. MacNeice, R. Rosner, J. Truran, and H. Tufo, "FLASH: An adaptive mesh hydrodynamics code for modeling astrophysical thermonuclear flashes," Astrophys. J. Suppl. S **131**, 273 (2000).
[67]B. T. Gunney and R. W. Anderson, "Advances in patch-based adaptive mesh refinement scalability," J. Parallel Distrib. Comput. **89**, 65–84 (2016).
[68]W. Zhang, A. Almgren, V. Beckner, J. Bell, J. Blaschke, C. Chan, M. Day, B. Friesen, K. Gott, D. Graves *et al.*, "AMReX: A framework for block-structured adaptive mesh refinement," J. Open Source Softw. **4**, 1370 (2019).
[69]W. Zhang, A. Myers, K. Gott, A. Almgren, and J. Bell, "AMReX: Block-structured adaptive mesh refinement for multiphysics applications," Int. J. High Performance Comput. Appl. **35**, 508–526 (2021).
[70]C. Burstedde, D. Calhoun, K. Mandli, and A. R. Terrel, "ForestClaw: Hybrid forest-of-octrees amr for hyperbolic conservation laws," in *Parallel Computing: Accelerating Computational Science and Engineering (CSE)* (IOS Press, 2014), pp. 253–262.
[71]P. Colella, D. T. Graves, T. Ligocki, D. Martin, D. Modiano, D. Serafini, and B. Van Straalen, "Chombo software package for AMR applications design document," Chombo **2**, 1–153 (2009), http://seesar.lbl.gov/ANAG/chombo/ (accessed September 2008).
[72]A. P. S. Bhalla, R. Bale, B. E. Griffith, and N. A. Patankar, "A unified mathematical framework and an adaptive numerical method for fluid–structure interaction with rigid, deforming, and elastic bodies," J. Comput. Phys. **250**, 446–476 (2013).
[73]Y. Zeng and L. Shen, "A unified AMR framework for multiphase flow and fluid-structure interaction problems with both non-subcycling and subcycling," in *Proceedings of the APS Division of Fluid Dynamics Meeting Abstracts* (APS, 2019), pp. S19–001.
[74]Y. Zeng, A. P. S. Bhalla, and L. Shen, "A subcycling/non-subcycling time advancement scheme-based dlm immersed boundary method framework for solving single and multiphase fluid–structure interaction problems on dynamically adaptive grids," Comput. Fluids **238**, 105358 (2022).
[75]A. P. S. Bhalla, R. Bale, B. E. Griffith, and N. A. Patankar, "Fully resolved immersed electrohydrodynamics for particle motion, electrolocation, and self-propulsion," J. Comput. Phys. **256**, 88–108 (2014).
[76]Y. Li and S.-C. Kong, "Mesh refinement algorithms in an unstructured solver for multiphase flow simulation using discrete particles," J. Comput. Phys. **228**, 6349–6360 (2009).
[77]N. Nangia, N. A. Patankar, and A. P. S. Bhalla, "A DLM immersed boundary method based wave-structure interaction solver for high density ratio multiphase flows," J. Comput. Phys. **398**, 108804 (2019).
[78]Y. Zeng, H. Liu, Q. Gao, A. Almgren, A. P. S. Bhalla, and L. Shen, "A consistent adaptive level set framework for incompressible two-phase flows with high density ratios and high Reynolds numbers," J. Comput. Phys. **478**, 111971 (2023).
[79]Y. Zeng, "Numerical simulations of the two-phase flow and fluid-structure interaction problems with adaptive mesh refinement," Ph.D. thesis (University of Minnesota, 2022).
[80]C. S. Peskin, "The immersed boundary method," Acta Numerica **11**, 479–517 (2002).
[81]A. J. Chorin, "The numerical solution of the Navier–Stokes equations for an incompressible fluid," Bull. Amer. Math. Soc. **73**, 928–931 (1967).
[82]M. Sussman, A. S. Almgren, J. B. Bell, P. Colella, L. H. Howell, and M. L. Welcome, "An adaptive level set approach for incompressible two-phase flows," J. Comput. Phys. **148**, 81–124 (1999).
[83]K. Sverdrup, N. Nikiforakis, and A. Almgren, "Highly parallelisable simulations of time-dependent viscoplastic fluid flow with structured adaptive mesh refinement," Phys. Fluids **30**, 093102 (2018).
[84]Y. Zeng, A. Xuan, J. Blaschke, and L. Shen, "A parallel cell-centered adaptive level set framework for efficient simulation of two-phase flows with subcycling and non-subcycling," J. Comput. Phys. **448**, 110740 (2022).
[85]A. S. Almgren, J. B. Bell, and W. G. Szymczak, "A numerical method for the incompressible Navier–Stokes equations based on an approximate projection," SIAM J. Sci. Comput. **17**, 358–369 (1996).
[86]W. J. Rider, "Approximate projection methods for incompressible flow: Implementation, variants and robustness," LANL Unclassified Report LA-UR-94–2000 (Los Alamos National Laboratory, 1995).
[87]E. Balaras and M. Vanella, "Adaptive mesh refinement strategies for immersed boundary methods," in Proceedings of the 47th AIAA Aerospace Sciences Meeting Including the New Horizons Forum and Aerospace Exposition (2009), p. 162.
[88]Z. Cui, Z. Yang, H.-Z. Jiang, W.-X. Huang, and L. Shen, "A sharp-interface immersed boundary method for simulating incompressible flows with arbitrarily deforming smooth boundaries," Int. J. Comput. Methods **15**, 1750080 (2018).
[89]S. He, Z. Yang, F. Sotiropoulos, and L. Shen, "Numerical simulation of interaction between multiphase flows and thin flexible structures," J. Comput. Phys. **448**, 110691 (2022).

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing

[90]B. E. Griffith, R. D. Hornung, D. M. McQueen, and C. S. Peskin, "An adaptive, formally second order accurate version of the immersed boundary method," J. Comput. Phys. **223**, 10–49 (2007).
[91]Z. Yao, R. Jambunathan, Y. Zeng, and A. Nonaka, "A massively parallel time-domain coupled electrodynamics–micromagnetics solver," Int. J. High Performance Comput. Appl. **36**, 167–181 (2022).
[92]A. Sharma, M. J. Brazell, G. Vijayakumar, S. Ananthan, L. Cheung, N. deVelder, M. T. Henry de Frahan, N. Matula, P. Mullowney, J. Rood *et al.*, "ExaWind: Open-source CFD for hybrid-RANS/LES geometry-resolved wind turbine simulations in atmospheric flows," Wind Energy **27**, 225–257 (2024).
[93]G. Pozzetti, H. Jasak, X. Besseron, A. Rousset, and B. Peters, "A parallel dual-grid multiscale approach to CFD–DEM couplings," J. Comput. Phys. **378**, 708–722 (2019).
[94]A. Myers, A. Almgren, L. D. Amorim, J. Bell, L. Fedeli, L. Ge, K. Gott, D. P. Grote, M. Hogan, A. Huebl *et al.*, "Porting WarpX to GPU-accelerated platforms," Parallel Comput. **108**, 102833 (2021).
[95]S. Tschisgale, T. Kempe, and J. Fröhlich, "A general implicit direct forcing immersed boundary method for rigid particles," Comput. Fluids **170**, 285–298 (2018).
[96]W.-Q. Wang, Y. Yan, and F.-B. Tian, "A simple and efficient implicit direct forcing immersed boundary model for simulations of complex flow," Appl. Math. Modell. **43**, 287–305 (2017).
[97]S. Tschisgale, T. Kempe, and J. Fröhlich, "A non-iterative immersed boundary method for spherical particles of arbitrary density ratio," J. Comput. Phys. **339**, 432–452 (2017).
[98]D. Young, Y. Lin, C. Fan, and C. Chiu, "The method of fundamental solutions for solving incompressible Navier–Stokes problems," Eng. Anal. Boundary Elem. **33**, 1031–1044 (2009).
[99]L. Schiller and A. Naumann, "On the basic calculations of gravity processing," Z. Ver. Ger. Ing. **77**, 318–321 (1933).
[100]Z. Gong, Z. Wu, C. An, B. Zhang, and X. Fu, "CP3d: A comprehensive Euler-Lagrange solver for direct numerical simulation of particle-laden flows," Comput. Phys. Commun. **286**, 108666 (2023).
[101]A. Ten Cate, C. Nieuwstad, J. J. Derksen, and H. Van den Akker, "Particle imaging velocimetry experiments and lattice-Boltzmann simulations on a single sphere settling under gravity," Phys. Fluids **14**, 4012–4025 (2002).
[102]S. V. Apte, M. Martin, and N. A. Patankar, "A numerical method for fully resolved simulation (FRS) of rigid particle–flow interactions in complex flows," J. Comput. Phys. **228**, 2712–2738 (2009).
[103]C.-C. Liao, W.-W. Hsiao, T.-Y. Lin, and C.-A. Lin, "Simulations of two sedimenting-interacting spheres with different sizes and initial configurations using immersed boundary method," Comput. Mech. **55**, 1191–1200 (2015).
[104]A. Yousefi, P. Costa, F. Picano, and L. Brandt, "On the role of inertia in channel flows of finite-size neutrally buoyant particles," J. Fluid Mech. **955**, A30 (2023).
[105]Y. Xia, H. Xiong, Z. Yu, and C. Zhu, "Effects of the collision model in interface-resolved simulations of particle-laden turbulent channel flows," Phys. Fluids **32**, 103303 (2020).
[106]P. Fan, Z. Lin, J. Xu, and Z. Yu, "A three-dimensional fictitious domain method for direct numerical simulations of particle-laden flows with heat transfer," Phys. Fluids **35**, 063324 (2023).
[107]G. Sharma, N. Nangia, A. P. S. Bhalla, and B. Ray, "A coupled distributed lagrange multiplier (DLM) and discrete element method (DEM) approach to simulate particulate flow with collisions," Powder Technol. **398**, 117091 (2022).
[108]Z. Zhu, R. Hu, and X. Zheng, "A multiple-time-step integration algorithm for particle-resolved simulation with physical collision time," Int. J. Multiphase Flow **163**, 104411 (2023).
[109]B. Wu, C. Shu, and M. Wan, "An implicit immersed boundary method for Robin boundary condition," Int. J. Mech. Sci. **261**, 108694 (2024).
[110]K. Sverdrup, A. Almgren, and N. Nikiforakis, "An embedded boundary approach for efficient simulations of viscoplastic fluids in three dimensions," Phys. Fluids **31**, 093102 (2019).
[111]P. Costa, B. J. Boersma, J. Westerweel, and W.-P. Breugem, "Collision model for fully resolved simulations of flows laden with finite-size particles," Phys. Rev. E **92**, 053012 (2015).
[112]J. Gan, Z. Zhou, and A. Yu, "CFD–DEM modeling of gas fluidization of fine ellipsoidal particles," AIChE. J. **62**, 62–77 (2016).

21 September 2026 07:31:41

Published under an exclusive license by AIP Publishing